DATA-260 Homework 6

Pinal Pawar | SID4 7875 | Domain: Grocery supply and recall notices
GitHub repository: https://github.com/PinalPawar/data260-7875
The repository link is working, and both collaborators (**Sbnikitha**, **supriyaselvanganesan**) have access. This submission is tagged **hw6**.

Section 0 - Personal Configuration

Value	Definition	This submission
SID4	Last four digits of SJSU Student ID	7875
PORT_BASE	8000 + (SID4 mod 900)	8675
PREFIX	"s" + SID4	s7875
SEED	SID4	7875
VERIFY_SEED	260000 + SID4	267875
DOMAIN_ID	SID4 mod 8	3 (Grocery supply and recall notices)
Hardware	Machine used for HW6	MacBook Air, Apple M2, 16 GB RAM (macOS); conda environment data260, Python 3.11
Local model	Ollama model used	qwen3:1.7b via Ollama (temperature 0, seed 7875); embeddings: sentence-transformers/ all-MiniLM-L6-v2
Tagged commit	Commit hash of tag hw6	5ded9db3c3210ac3c34629b6a473998274e19049

Part 1 - MongoDB Document API

Part 1 adds a document-oriented view of the recall-notice domain, stored in MongoDB alongside the existing MySQL tables from HW4/HW5. The code lives in a new root-level folder `mongo_doc/` and is plugged into the existing `main.py`, so the same FastAPI app on port 8675 serves both the MySQL and the MongoDB endpoints.

I. Database

MongoDB runs locally in Docker (container `s7875_mongo`, image `mongo:7.0`, port 27017). The database is **s7875_doc** and the collection is **recall_notices**. Each document is one recall notice. The generic fields required by the assignment map to the domain like this:

Assignment requirement	Field	Rule
Required short text (max 100)	product	required, 1-100 characters

Assignment requirement	Field	Rule
Optional description	description	optional, up to 2000 characters
Status-style enum with default	status	announced / ongoing / resolved, default announced
Category-style fixed options	category	supplyShortage / bacterialContamination / foreignMaterial / mislabelling
Required date/time	recall_date	required, ISO 8601 date-time
Automatic timestamps	createdAt, updatedAt	set by the server on insert and update

Validation is enforced in two places: the Pydantic schemas reject bad requests before they reach the database (Section III), and a MongoDB `$jsonSchema` validator installed on the collection refuses any invalid document written by another client. The validator is created on server startup:

mongo_doc/db.py - MongoDB-side $jsonSchema validator (excerpt)

```
NOTICE_VALIDATOR = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["product", "status", "category", "recall_date",
                     "createdAt", "updatedAt"],
        "properties": {
            "product": {"bsonType": "string", "minLength": 1, "maxLength": 100,
                        "description": "required short text, max 100 chars"},
            "description": {"bsonType": ["string", "null"],
                            "description": "optional longer description"},
            "status": {"enum": STATUS_VALUES,
                       "description": "recall status, default 'announced'"},
            "category": {"enum": CATEGORY_VALUES,
                         "description": "fixed set of recall categories"},
            "recall_date": {"bsonType": "date",
                            "description": "required date the recall was issued"},
            "createdAt": {"bsonType": "date"},
            "updatedAt": {"bsonType": "date"},
        },
```

```
    }
}
```

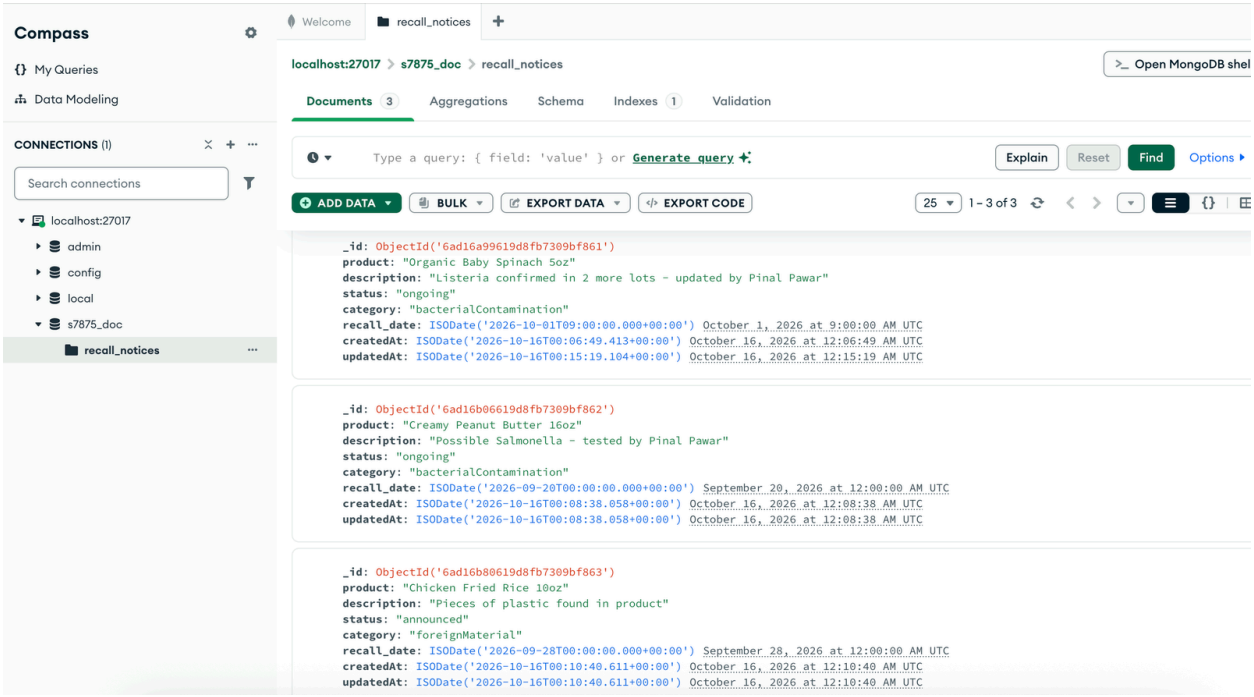


Figure 1.1 - MongoDB Compass: s7875_doc > recall_notices with sample documents

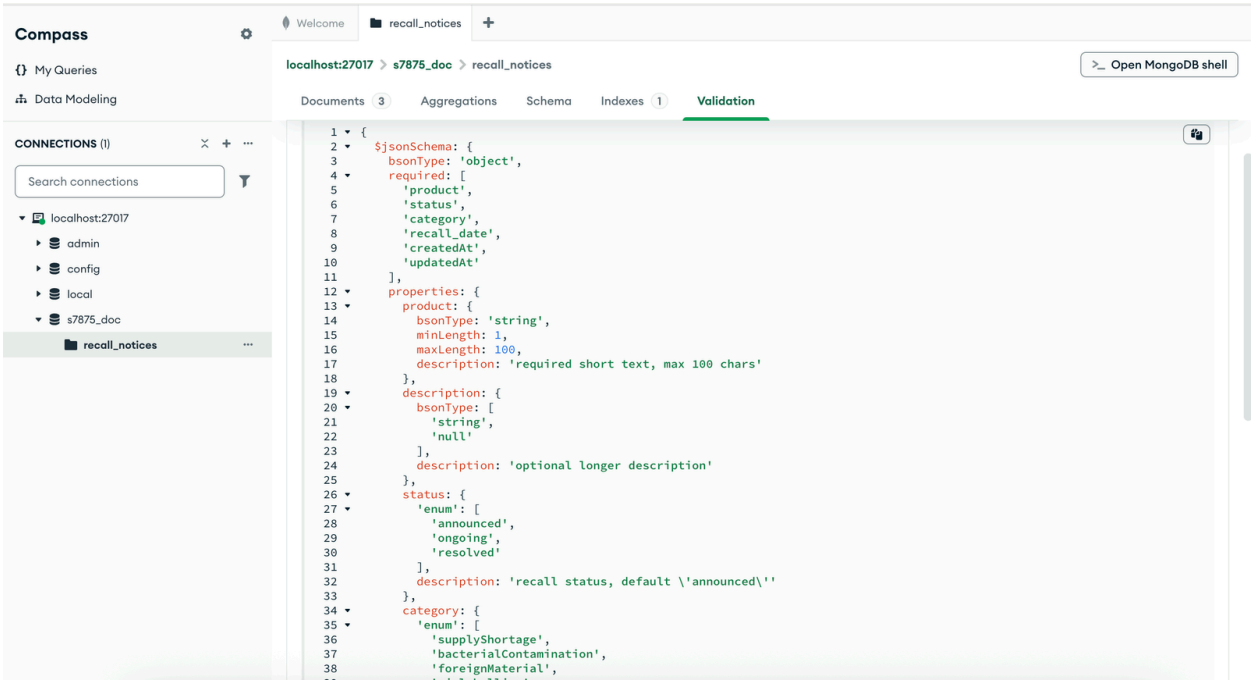


Figure 1.2 - MongoDB Compass Validation tab: required fields, string length and enum rules

II. API Implementation

The code is split into separate files, as required:

File	Responsibility
mongo_doc/config.py	Server configuration: MongoDB URI, database and collection names (from environment / .env)
mongo_doc/db.py	PyMongo connection (one shared client) and the $jsonSchema validator
mongo_doc/schemas.py	Pydantic models: NoticeDocCreate, NoticeDocUpdate, NoticeDocOut
mongo_doc/routes/ notice_docs.py	Route handlers for the five CRUD endpoints and error handling
main.py	Existing app: CORS middleware, includes the new router, installs the validator at startup

Middleware. CORS is handled by the existing `CORSMiddleware` in `main.py`. JSON request bodies are parsed by FastAPI automatically: each handler declares a Pydantic model as its parameter, and FastAPI reads the JSON body into that model. The new router is registered in `main.py`:

main.py - middleware and router registration (excerpt)

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
...
# HW6 Part 1: MongoDB document API (database s7875_doc, collection
# recall_notices). Lives in mongo_doc/ with its own config, db, schemas
# and routes files. FastAPI parses JSON request bodies automatically.
app.include_router(notice_docs_router)



@app.on_event("startup")
```

```
def _hw6_init_mongo():
    try:
        init_mongo_collection()
    except Exception as exc:
        print(f"[HW6] MongoDB init skipped: {exc}")
```

Error handling. Every endpoint returns a specific status code for each failure case:

Situation	Status code
Record created	201 Created
Record read or updated	200 OK
Record deleted	204 No Content
Id is not a valid MongoDB ObjectId, or PUT with no fields	400 Bad Request
No record with that id	404 Not Found
Request body fails Pydantic validation	422 Unprocessable Entity
MongoDB not reachable	503 Service Unavailable
Any other database error	500 Internal Server Error

mongo_doc/routes/notice_docs.py - shared error helpers

```
def _oid(notice_id: str) -> ObjectId:
    try:
        return ObjectId(notice_id)
    except (InvalidId, TypeError):
        raise HTTPException(status_code=400,
                            detail=f"'{notice_id}' is not a valid MongoDB id.")


def _db_error(exc: PyMongoError, action: str):
    if isinstance(exc, ServerSelectionTimeoutError):
        raise HTTPException(status_code=503,
                            detail="MongoDB is not reachable. Is the container running?")
    raise HTTPException(status_code=500, detail=f"Database error while {action}.")
```

Create (POST /api/doc/notices). Inserts the validated body and adds both timestamps:

Create handler

```
@router.post("", response_model=NoticeDocOut, status_code=status.HTTP_201_CREATED)
def create_notice_doc(payload: NoticeDocCreate):
    doc = payload.model_dump()
    doc["createdAt"] = doc["updatedAt"] = _now()
    try:
        col = get_notices_collection()
        res = col.insert_one(doc)
        return _serialize(col.find_one({"_id": res.inserted_id}))
    except PyMongoError as exc:
        _db_error(exc, "creating the notice")
```

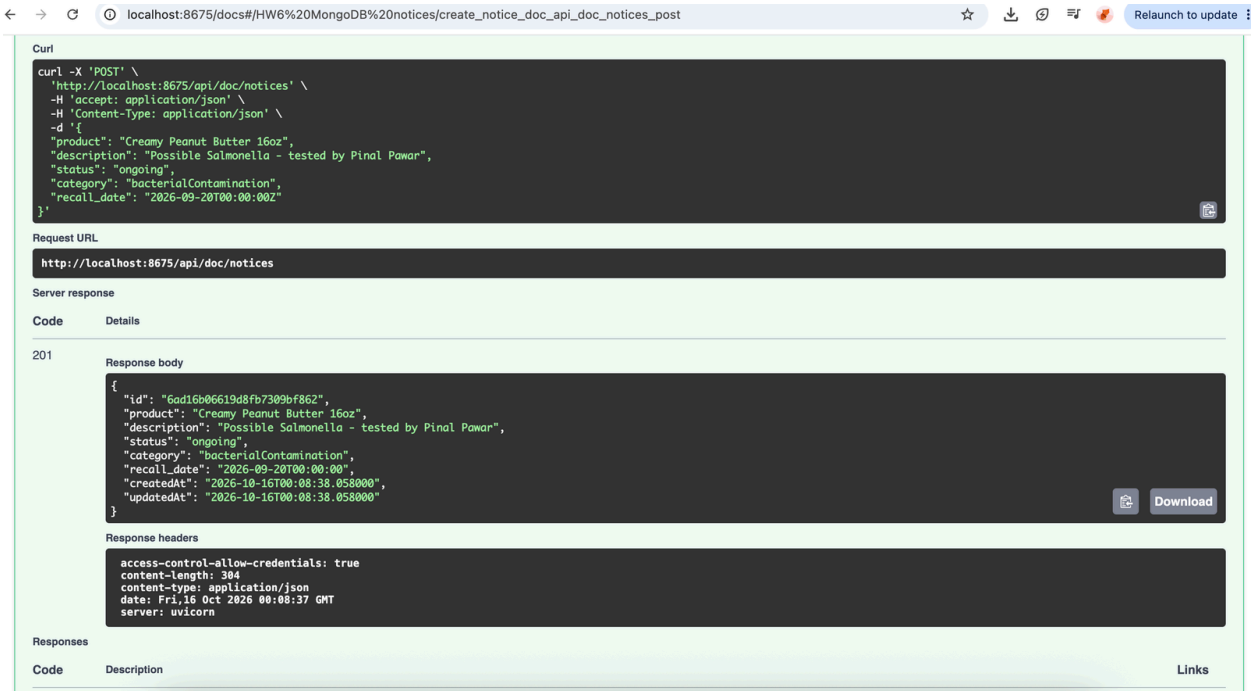


Figure 1.3 - POST creates a notice: 201 Created, with id and timestamps

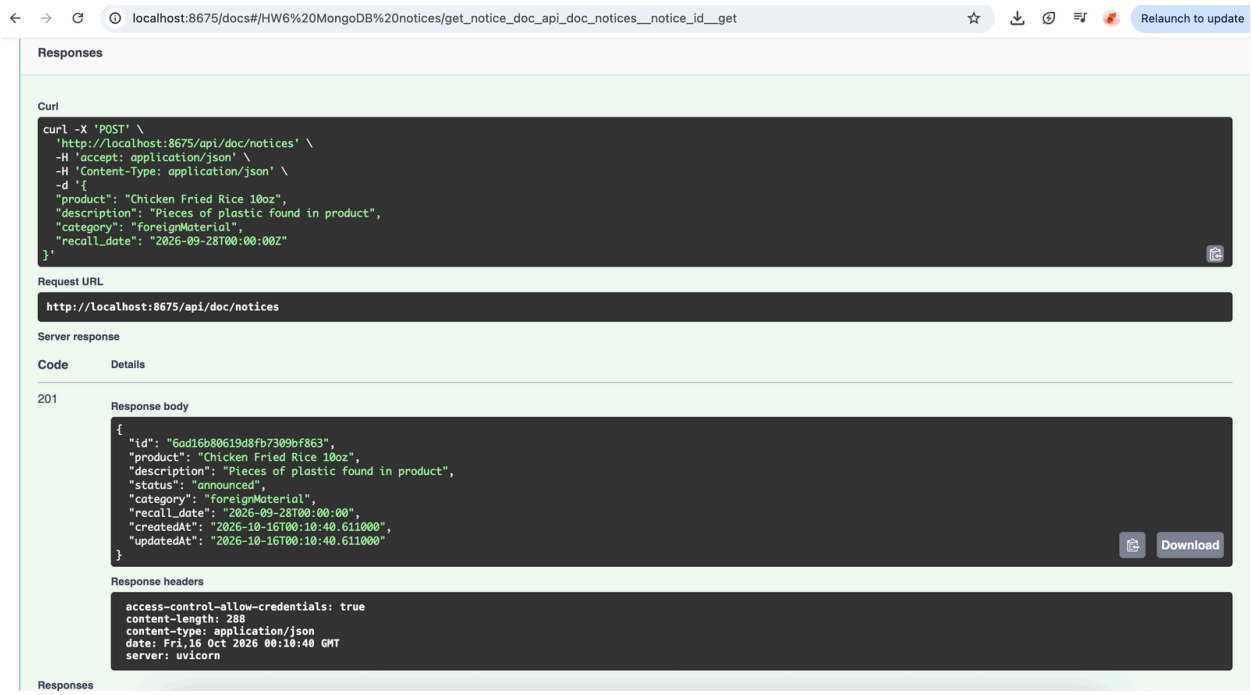


Figure 1.4 - POST without a status: the default "announced" is applied

Retrieve all (GET /api/doc/notices). Returns every notice, newest first:

List handler

```
@router.get("", response_model=list[NoticeDocOut])
def list_notice_docs():
    try:
        docs = get_notices_collection().find().sort("createdAt", -1)
        return [_serialize(d) for d in docs]
    except PyMongoError as exc:
        _db_error(exc, "listing notices")
```

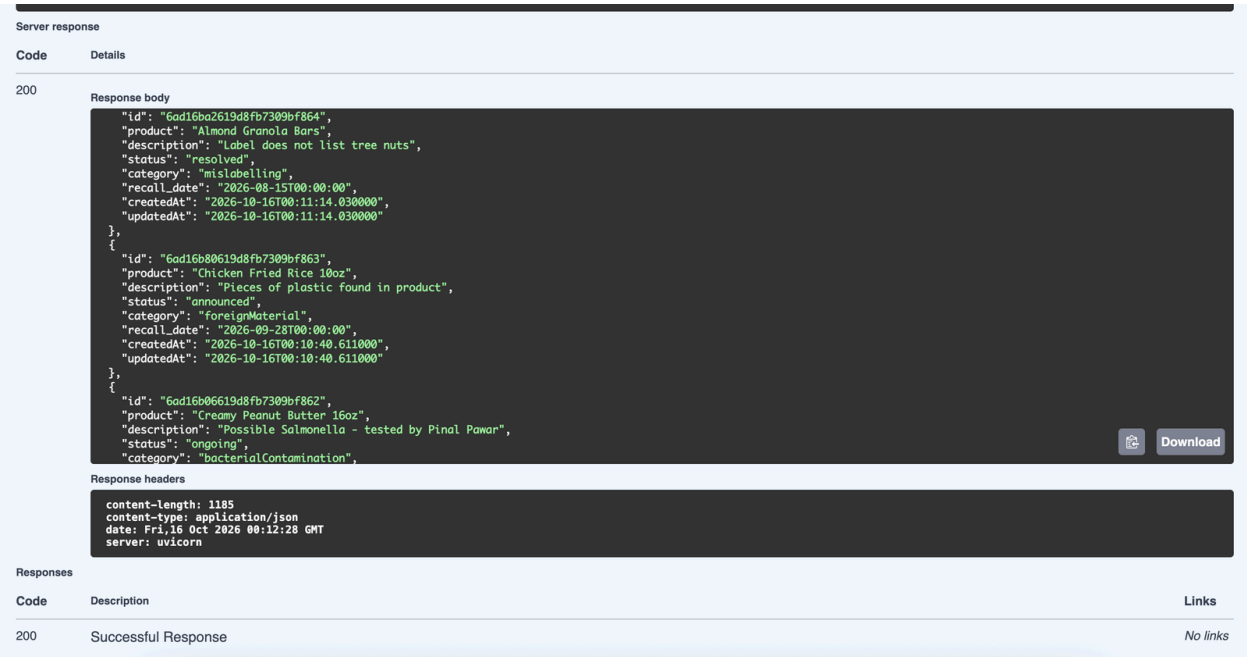


Figure 1.5 - GET all notices: 200 OK

Retrieve one (GET /api/doc/notices/{id}):

Get-by-id handler

```
@router.get("/{notice_id}", response_model=NoticeDocOut)
def get_notice_doc(notice_id: str):
    oid = _oid(notice_id)
    try:
        doc = get_notices_collection().find_one({"_id": oid})
    except PyMongoError as exc:
        _db_error(exc, "fetching the notice")
    if doc is None:
        raise HTTPException(status_code=404, detail="Notice not found.")
    return _serialize(doc)
```

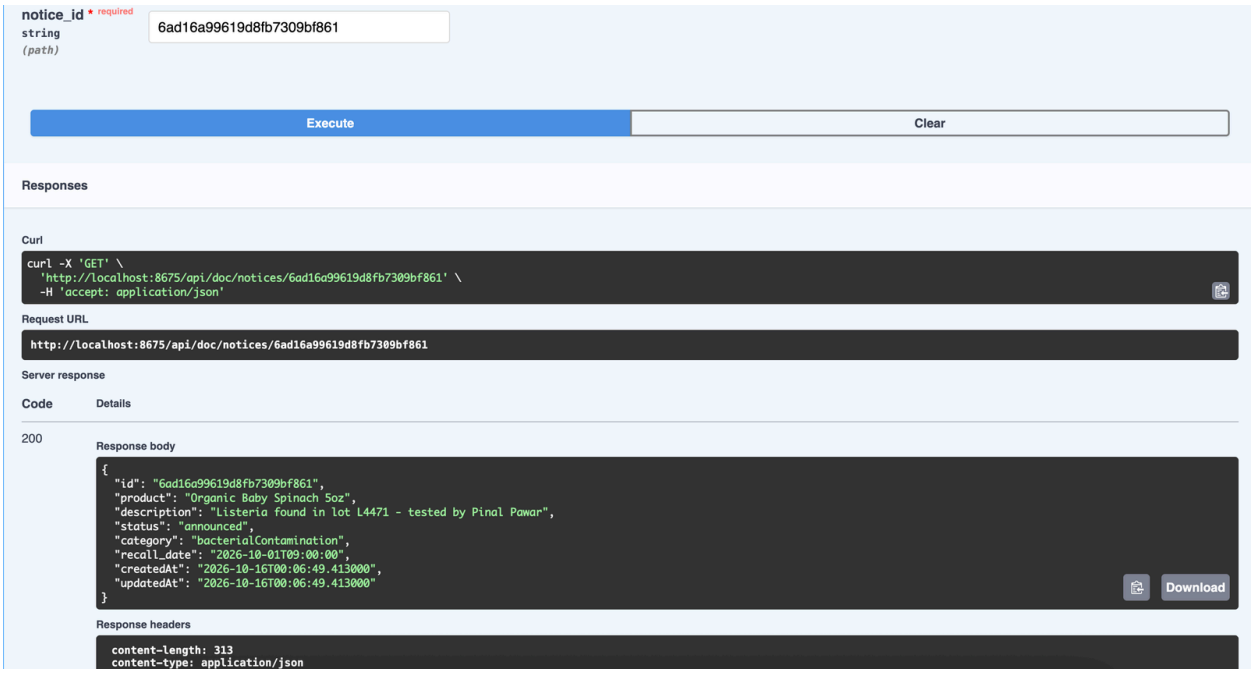


Figure 1.6 - GET one notice by id: 200 OK

Update (PUT /api/doc/notices/{id}). Only the fields sent are changed, and `updatedAt` is refreshed:

Update handler

```
@router.put("/{notice_id}", response_model=NoticeDocOut)
def update_notice_doc(notice_id: str, payload: NoticeDocUpdate):
    oid = _oid(notice_id)
    updates = payload.model_dump(exclude_unset=True)
    if not updates:
        raise HTTPException(status_code=400, detail="No fields provided to update.")
    updates["updatedAt"] = _now()
    try:
        doc = get_notices_collection().find_one_and_update(
            {"_id": oid}, {"$set": updates}, return_document=ReturnDocument.AFTER)
    except PyMongoError as exc:
        _db_error(exc, "updating the notice")
    if doc is None:
        raise HTTPException(status_code=404, detail="Notice not found.")
    return _serialize(doc)
```

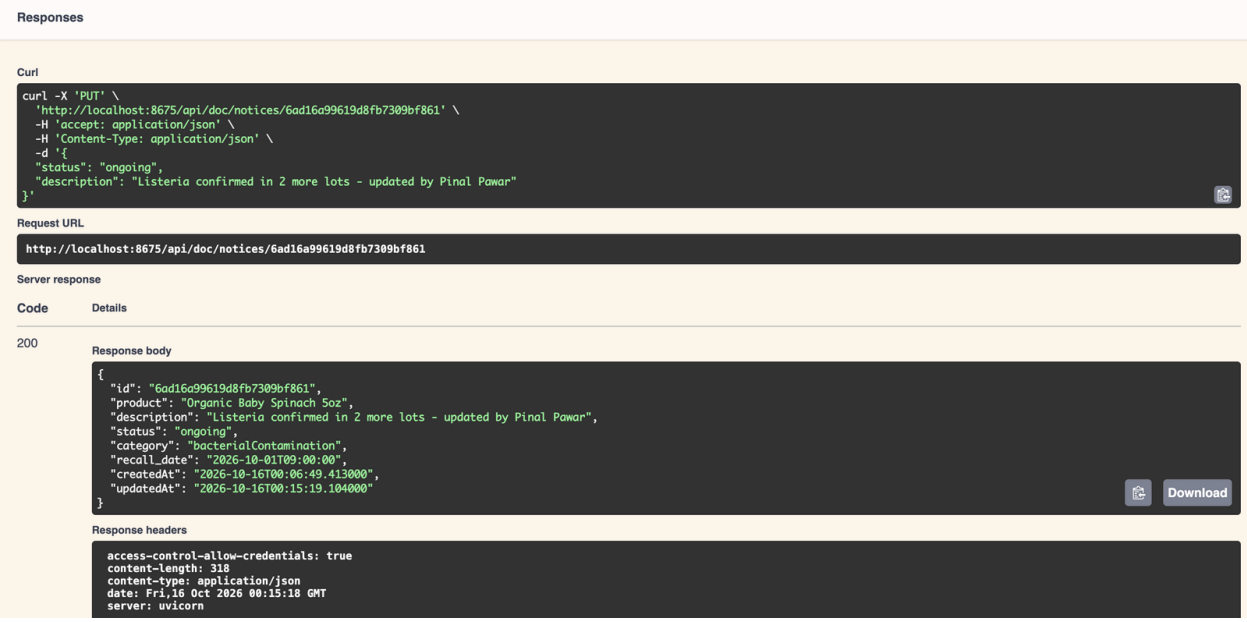


Figure 1.7 - PUT updates status and description: 200 OK; updatedAt is now later than createdAt

Delete (DELETE /api/doc/notices/{id}):

Delete handler

```
@router.delete("/{notice_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_notice_doc(notice_id: str):
    oid = _oid(notice_id)
    try:
        res = get_notices_collection().delete_one({"_id": oid})
    except PyMongoError as exc:
        _db_error(exc, "deleting the notice")
    if res.deleted_count == 0:
        raise HTTPException(status_code=404, detail="Notice not found.")
    return Response(status_code=204)
```



Figure 1.8 - DELETE: 204 No Content

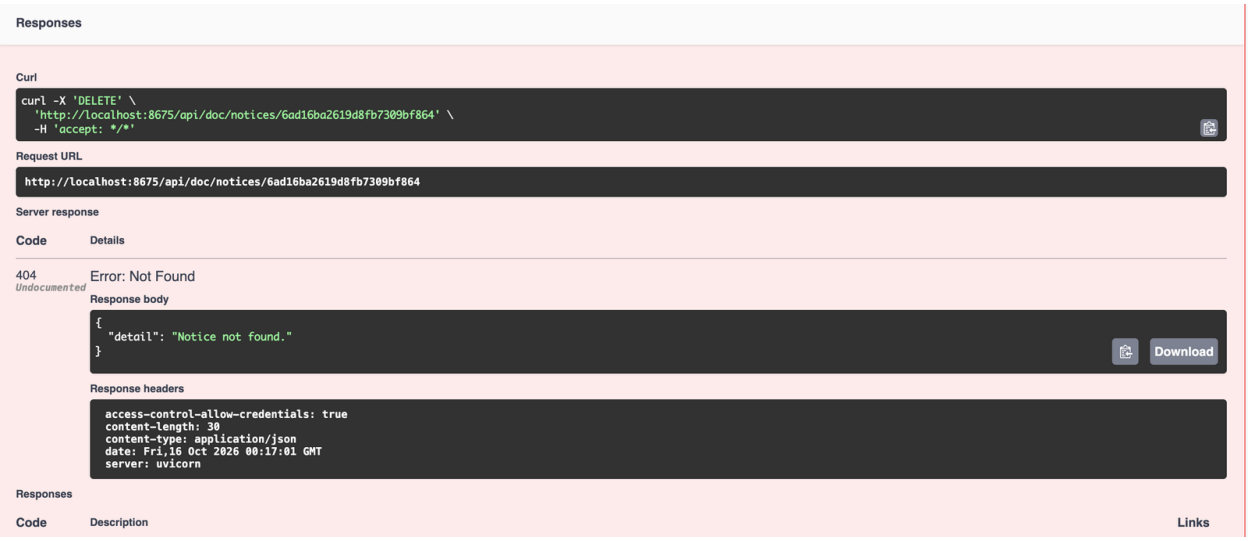


Figure 1.9 - DELETE the same id again: 404 "Notice not found."

III. Data Validation

The Pydantic schemas enforce required fields, string lengths, enum values and date formats. `extra="forbid"` also rejects unknown fields instead of silently storing them. The update model makes every field optional so PUT can change a single field.

mongo_doc/schemas.py - create model (the update model makes every field optional)

```
class NoticeDocCreate(BaseModel):

    # extra="forbid": unknown fields are rejected instead of silently stored

    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)


    product: str = Field(..., min_length=1, max_length=100)

    description: Optional[str] = Field(None, max_length=2000)
```

```
status: StatusEnum = "announced"

category: CategoryEnum

recall_date: datetime
```

Each invalid request below was rejected before reaching MongoDB, with a clear message naming the field and the rule it broke:

Test	Invalid input	Result
Invalid enum value	category = "spoiled"	422, lists the four allowed categories
Missing required fields	only category sent	422, "Field required" for product and recall_date
Incorrect data type	recall_date = "yesterday"	422, "Input should be a valid datetime"
String too long	product over 100 characters	422, "String should have at most 100 characters"
Invalid id format	GET /api/doc/notices/abc	400, "'abc' is not a valid MongoDB id."

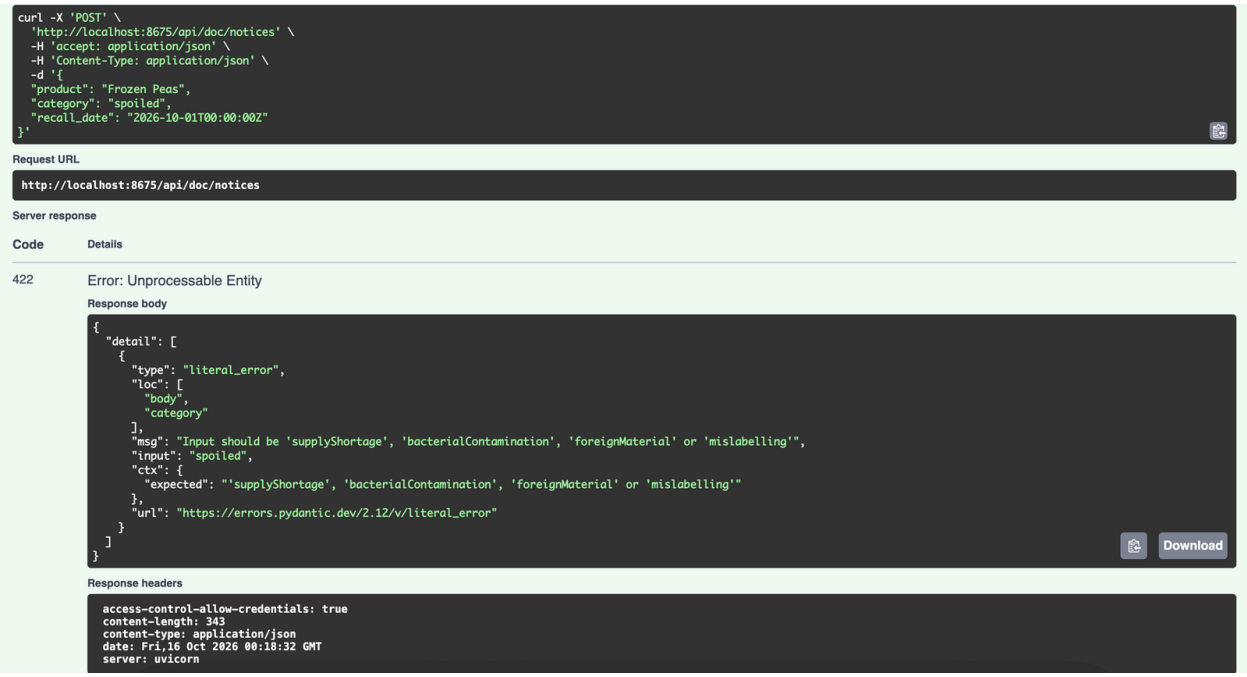


Figure 1.10 - Invalid enum value: 422

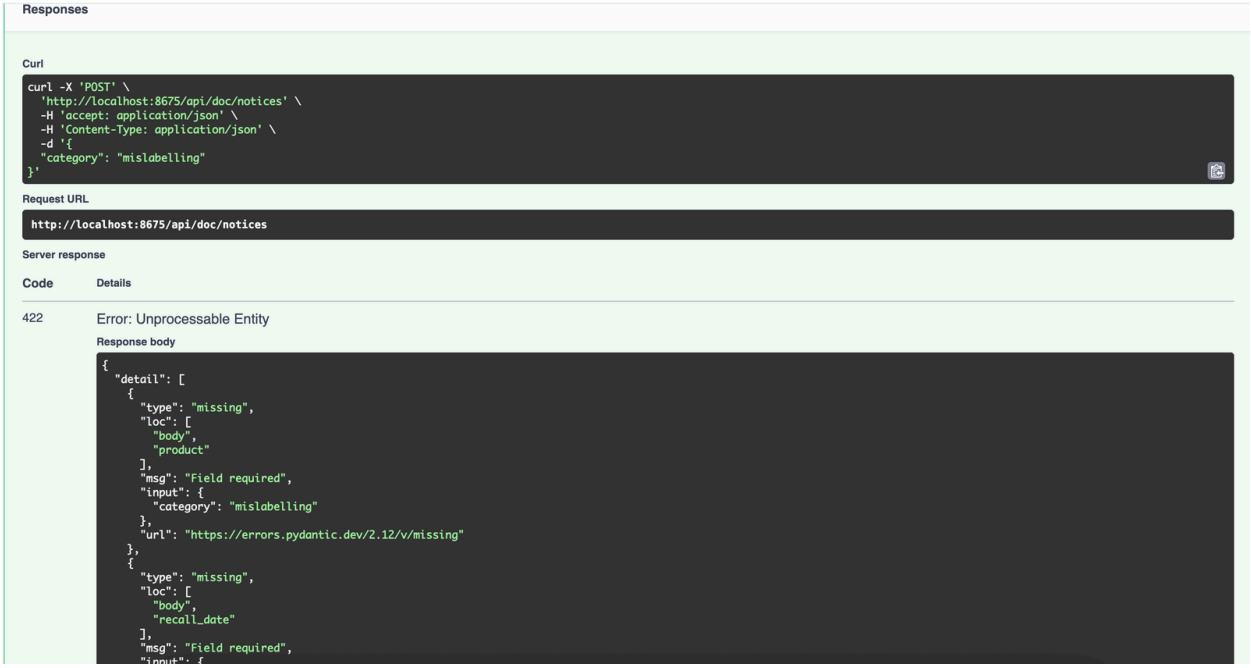


Figure 1.11 - Missing required fields: 422

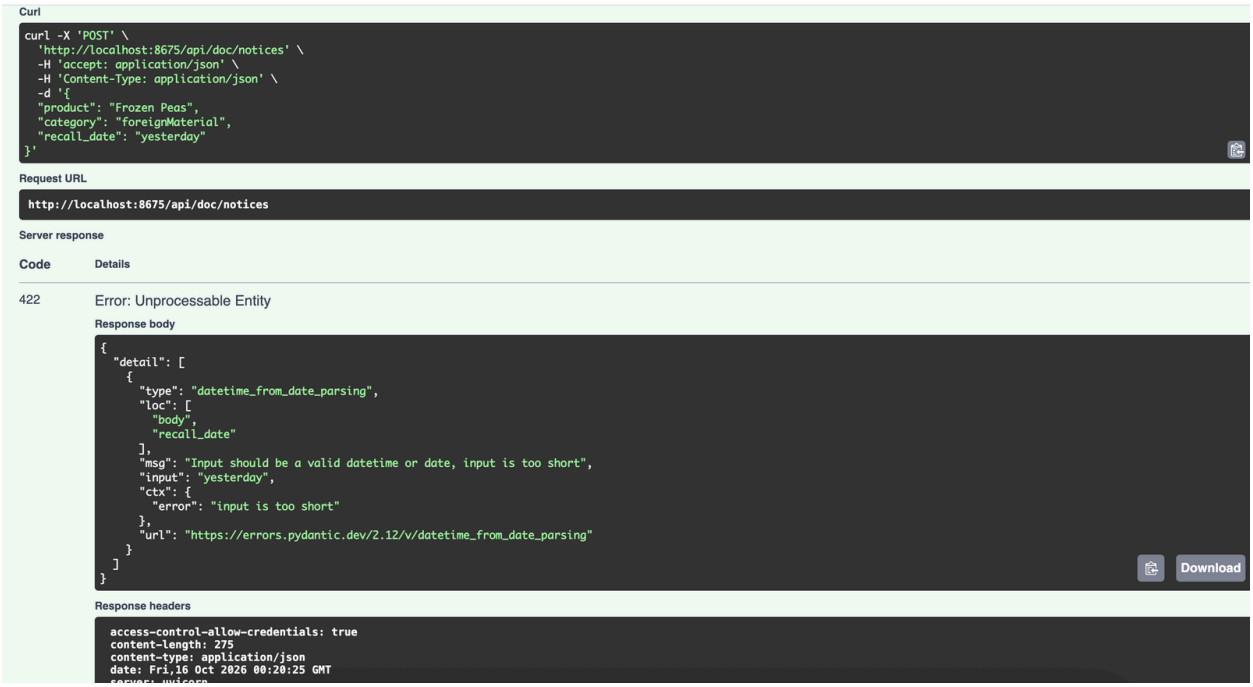


Figure 1.12 - Incorrect data type for recall_date: 422

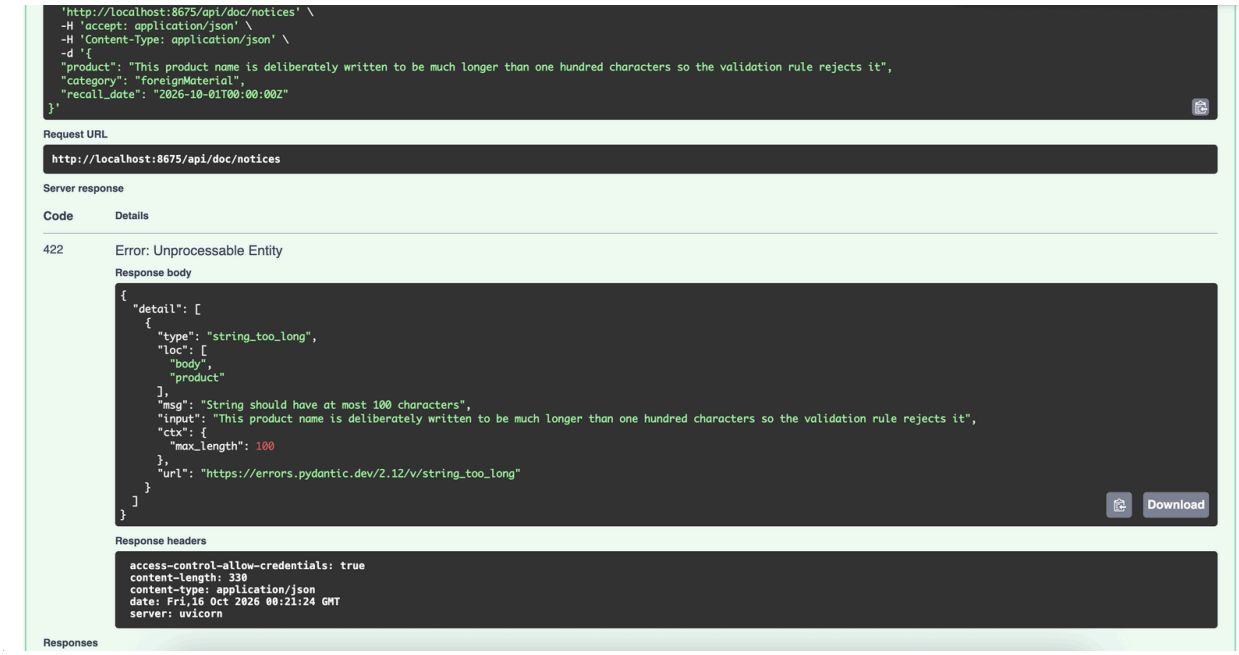


Figure 1.13 - product longer than 100 characters: 422

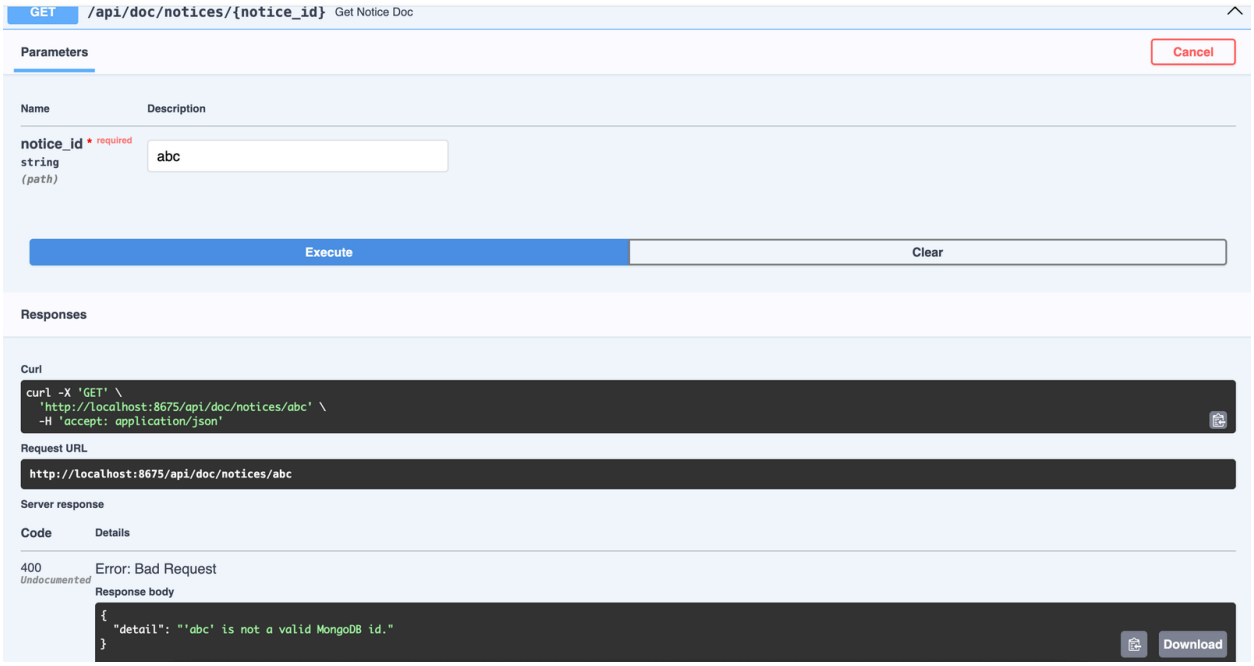


Figure 1.14 - Invalid id format: 400

Part 2 - Short-, Long-, and Episodic Memory

Part 2 adds a chat assistant, **RecallHelper**, with three kinds of memory. All three live in the same `s7875_doc` database as three new collections, so the Part 1 `recall_notices` collection is untouched and only the connection is reused. The model is the local Ollama model **qwen3:1.7b**, always called through the HW1 adapter `src/model_client.complete()`. Embeddings use `sentence-`

`transformers/all-MiniLM-L6-v2` (384 dimensions), the same embedding model used for retrieval in HW3/HW4.

MongoDB collections

Collection	Fields	Memory type
messages	user_id, session_id, role (user/assistant), content, created_at	Short-term: sliding window of the last N messages
summaries	user_id, session_id (null for lifetime), scope ('session' | 'user'), text, created_at	Long-term: session and lifetime summaries
episodes	user_id, session_id, fact, importance (0..1), embedding (array of 384 floats), created_at	Episodic: facts retrieved by cosine similarity

Each collection is created at startup with a `$jsonSchema` validator (for example `role` must be user/assistant, `scope` must be session/user, `importance` must be between 0 and 1) and an index that matches its main query:

mongo_doc/db.py - collection setup

```
def init_memory_collections():
    """Create messages / summaries / episodes with validators and the
    indexes the memory queries use. Leaves recall_notices untouched."""
    _create_or_update(MESSAGES_COLLECTION, MESSAGES_VALIDATOR)
    _create_or_update(SUMMARIES_COLLECTION, SUMMARIES_VALIDATOR)
    _create_or_update(EPISODES_COLLECTION, EPISODES_VALIDATOR)
    get_messages_collection().create_index(
        [("user_id", ASCENDING), ("session_id", ASCENDING), ("created_at", ASCENDING)])
    get_summaries_collection().create_index(
        [("user_id", ASCENDING), ("scope", ASCENDING), ("created_at", DESCENDING)])
    get_episodes_collection().create_index(
        [("user_id", ASCENDING), ("created_at", DESCENDING)])
```

Setting	Value	Meaning
SHORT_TERM_N	12	messages of the current session placed in the prompt
SUMMARIZE_EVERY_USER_MSGS	4	a session summary is written every 4 user messages

Setting	Value	Meaning
LIFETIME_EVERY_SESSION_SUMMARIES	2	every 2 session summaries are condensed into a lifetime summary
EPISODIC_TOP_K	3	episodic facts retrieved per turn
MAX_FACTS_PER_MESSAGE	3	facts extracted from each user message

Endpoint 1 - POST /api/chat

One request runs the full memory cycle: read memory, call the model, write memory. The response returns the reply plus everything that was used (short-term count and list, both summaries, the retrieved facts) and everything that was written.

mongo_doc/routes/chat.py - POST /api/chat

```
@router.post("/chat")
def chat(req: ChatRequest,
         short_term_n: Optional[int] = Query(None, ge=1, le=100,
             description="Experiment override for SHORT_TERM_N (Part 3)"),
         policy: Literal["none", "compress"] = Query("none",
             description="Memory-saving policy for the short-term window (Part 3)")):
    t0 = time.perf_counter()
    n = short_term_n or SHORT_TERM_N
    session_id = req.session_id or uuid.uuid4().hex[:12]
    memory.start_call_log()
    try:
        # 1. save the user message
        user_doc = memory.save_message(req.user_id, session_id, "user", req.message)
        # 2. short-term window: last N messages before this one
        window = memory.load_short_term(req.user_id, session_id, n,
                                        before_id=user_doc["_id"])
        prompt_window = memory.compress_window(window) if policy == "compress" else window
        # 3. latest session summary + lifetime summary
        session_summary = memory.latest_session_summary(req.user_id, session_id)
        lifetime_summary = memory.latest_lifetime_summary(req.user_id)
        # 4. top-k episodic facts by cosine similarity to this message
```

```
        query_vector = memory.embed([req.message])[0]
        facts = memory.retrieve_episodes(req.user_id, query_vector, EPISODIC_TOP_K)
        # 5. compose the prompt
        messages, parts = memory.build_prompt(
            req.message, prompt_window,
            session_summary["text"] if session_summary else None,
            lifetime_summary["text"] if lifetime_summary else None,
            facts)
    except PyMongoError as exc:
        _db_error(exc)

    # 6. call the local model through the HW1 adapter
    try:
        reply, usage = memory.call_model(messages)
    except Exception as exc:
        raise HTTPException(status_code=502,
                            detail=f"Local model (Ollama) call failed: {exc}")
    if not reply:
        raise HTTPException(status_code=502, detail="The model returned an empty reply.
Please try again.")

    try:
        # 7. save the assistant reply
        memory.save_message(req.user_id, session_id, "assistant", reply)
    except PyMongoError as exc:
        _db_error(exc)

    # 8 + 9 are background memory writes: if one fails (e.g. a model timeout)
    # the user still gets the reply, and the error is reported in the response.
    memory_errors = []
    try:
        # 8. extract new episodic facts from this message
        new_facts = memory.extract_episodes(req.user_id, session_id, req.message)
```

```
    except Exception as exc:
        new_facts = []
        memory_errors.append(f"episode extraction failed: {exc}")
    try:
        # 9. summarization trigger (session every M user msgs, lifetime occasionally)
        summary_events = memory.maybe_summarize(req.user_id, session_id)
    except Exception as exc:
        summary_events = {"session_summary_updated": False, "lifetime_summary_updated":
False}
        memory_errors.append(f"summarization failed: {exc}")

    # token cost of this turn: the reply call, plus memory upkeep calls
    calls = memory.get_call_log()
    upkeep = [c for c in calls if c["purpose"] != "reply"]
    parts_tokens = {k: memory.count_tokens(v) for k, v in parts.items()}
    total_ms = round((time.perf_counter() - t0) * 1000, 1)
    _log_request({
        "ts": datetime.now(timezone.utc).isoformat(),
        "user_id": req.user_id, "session_id": session_id, "model": CHAT_MODEL,
        "short_term_n": n, "policy": policy, "window_count": len(window),
        "prompt_tokens_ollama": usage["prompt_tokens"],
        "prompt_tokens": usage["prompt_tokens_full"],
        "completion_tokens": usage["completion_tokens"],
        **{f"tokens_{k}": v for k, v in parts_tokens.items()},
        "reply_latency_ms": usage["latency_ms"], "request_latency_ms": total_ms,
        "upkeep_calls": len(upkeep),
        "upkeep_prompt_tokens": sum(c["prompt_tokens_full"] for c in upkeep),
        "upkeep_completion_tokens": sum(c["completion_tokens"] for c in upkeep),
        "upkeep_latency_ms": round(sum(c["latency_ms"] for c in upkeep), 1),
        "session_summary_updated": summary_events["session_summary_updated"],
        "lifetime_summary_updated": summary_events["lifetime_summary_updated"],
        "message": req.message, "reply": reply,
    })
```

```
    return {
        "reply": reply,
        "user_id": req.user_id,
        "session_id": session_id,
        "memory_used": {
            "short_term": {
                "count": len(window),
                "n_limit": n,
                "policy": policy,
                "messages": [{"role": m["role"], "content": m["content"]} for m in window],
            },
            "session_summary": session_summary["text"] if session_summary else None,
            "lifetime_summary": lifetime_summary["text"] if lifetime_summary else None,
            "episodic_facts": facts,
        },
        "memory_written": {"new_episodic_facts": new_facts, **summary_events,
                           "errors": memory_errors},
        "usage": {**usage, "total_request_ms": total_ms,
                  "upkeep_calls": upkeep},
        "prompt_parts_tokens": parts_tokens,
    }
```

Memory logic

Short-term memory is a sliding window: the last `SHORT_TERM_N` messages of the session, oldest first. The message just saved is excluded because it is added to the prompt separately.

Short-term window

```
def load_short_term(user_id, session_id, n=SHORT_TERM_N, before_id=None):
    """Last n messages of this session, oldest first. `before_id` excludes the
    message just saved, because the current message is added to the prompt
    separately."""
    query = {"user_id": user_id, "session_id": session_id}
```

```
    if before_id is not None:
        query["_id"] = {"$ne": before_id}
    docs = list(get_messages_collection().find(query)
                .sort([("created_at", DESCENDING), ("_id", DESCENDING)]).limit(n))
    return list(reversed(docs))
```

Summarization trigger. After every turn the server counts the user messages in the session. Every 4th one triggers a session summary (scope='session'); each new session summary builds on the previous one, so nothing older is lost when messages slide out of the window. Every 2nd session summary triggers a lifetime summary (scope='user', session_id=null) that condenses the latest summary of each recent session.

Summarization trigger

```
def maybe_summarize(user_id, session_id):
    """Summarization trigger, run after each turn.
    - every SUMMARIZE_EVERY_USER_MSGS user messages in the session -> session summary
    - every LIFETIME_EVERY_SESSION_SUMMARIES session summaries of the user -> lifetime
summary"""
    result = {"session_summary_updated": False, "lifetime_summary_updated": False}
    user_msgs = get_messages_collection().count_documents(
        {"user_id": user_id, "session_id": session_id, "role": "user"})
    if user_msgs == 0 or user_msgs % SUMMARIZE_EVERY_USER_MSGS != 0:
        return result
    summarize_session(user_id, session_id)
    result["session_summary_updated"] = True
    n_session_summaries = get_summaries_collection().count_documents(
        {"user_id": user_id, "scope": "session"})
    if n_session_summaries % LIFETIME_EVERY_SESSION_SUMMARIES == 0:
        condense_lifetime_summary(user_id)
        result["lifetime_summary_updated"] = True
    return result
```

Session summary

```
def summarize_session(user_id, session_id):
```

```
    """Summarize the recent conversation (plus the previous session summary,
    so nothing older is lost) into concise bullets, scope='session'."""
    previous = latest_session_summary(user_id, session_id)
    recent = load_short_term(user_id, session_id, n=2 * SUMMARIZE_EVERY_USER_MSGS)
    prompt = (
        "Summarize this conversation between a user and a grocery-recall "
        "assistant into 3-6 concise bullet points. Each line must start with "
        "'- '. Keep facts about the user (name, location, family, allergies, "
        "products they bought, stores) and any recalls discussed. No preamble.\n\n"
        + (f"Earlier summary:\n{previous['text']}\n\n" if previous else "")
        + f"Recent messages:\n{_transcript(recent)}"
    )
    text, usage = call_model([{"role": "user", "content": prompt + " /no_think"}],
                             purpose="session_summary")
    if not text:
        raise RuntimeError("model returned an empty session summary; not stored")
    doc = {"user_id": user_id, "session_id": session_id, "scope": "session",
           "text": text, "created_at": now()}
    get_summaries_collection().insert_one(doc)
    return doc, usage
```

Lifetime summary

```
def condense_lifetime_summary(user_id):
    """Condense the latest summary of each recent session (plus the previous
    lifetime summary) into one lifetime summary: scope='user', session_id=None."""
    col = get_summaries_collection()
    pipeline = [
        {"$match": {"user_id": user_id, "scope": "session"}},
        {"$sort": {"created_at": -1}},
        {"$group": {"_id": "$session_id", "text": {"$first": "$text"},
                    "created_at": {"$first": "$created_at"}}},
        {"$sort": {"created_at": -1}},
```

```
        {"$limit": 6},
    ]
    sessions = [x for x in col.aggregate(pipeline) if x.get("text")]
    if not sessions:
        raise RuntimeError("no session summaries to condense")
    previous = latest_lifetime_summary(user_id)
    prompt = (
        "Condense these notes about one user into a single lifetime profile of "
        "4-8 bullet points ('- ' each). Merge duplicates. If two notes "
        "conflict, keep the most recent one. No preamble.\n\n"
        + (f"Previous lifetime profile:\n{previous['text']}\n\n" if previous else "")
        + "\n\n".join(f"Session notes:\n{s['text']}" for s in reversed(sessions))
    )
    text, usage = call_model([{"role": "user", "content": prompt + " /no_think"}],
                             max_tokens=400, purpose="lifetime_summary")
    if not text:
        raise RuntimeError("model returned an empty lifetime summary; not stored")
    doc = {"user_id": user_id, "session_id": None, "scope": "user",
           "text": text, "created_at": now()}
    col.insert_one(doc)
    return doc, usage
```

Episodic memory. The model extracts up to 3 short facts from each user message, each with an importance score. Each fact is embedded and stored with its 384-number vector. A new fact whose cosine similarity to a stored fact is 0.92 or higher is treated as a duplicate and skipped.

Episode extraction

```
def extract_episodes(user_id, session_id, message):
    """Ask the model for up to 3 short, durable facts in the user's message,
    embed each one and store it. Near-duplicates of stored facts are skipped."""
    prompt = (
        "Extract up to 3 short facts worth remembering about the USER from the "
        "message below (who they are, where they live or shop, family, "
        "allergies or diet, products they bought, preferences). Ignore greetings "
```

```
        "and plain questions. Give each fact an importance from 0 to 1 "
        "(1 = safety-critical, e.g. an allergy or a recalled product at home).\n"
        'Reply with ONLY a JSON list, e.g. [{"fact": "User is allergic to peanuts", '
        '"importance": 0.9}]. Reply [] if there is nothing to remember.\n\n'
        f"Message: {message} /no_think"
    )
    text, _ = call_model([{"role": "user", "content": prompt}], max_tokens=200,
                         purpose="episode_extraction")
    facts = _parse_facts(text)
    if not facts:
        return []
    vectors = embed([f["fact"] for f in facts])
    col = get_episodes_collection()
    existing = list(col.find({"user_id": user_id}, {"embedding": 1}))
    stored = []
    for f, vec in zip(facts, vectors):
        if any(cosine(vec, e["embedding"]) >= EPISODE_DUPLICATE_SIMILARITY for e in
existing):
            continue
        doc = {"user_id": user_id, "session_id": session_id, "fact": f["fact"],
               "importance": f["importance"], "embedding": vec, "created_at": now()}
        col.insert_one(doc)
        existing.append(doc)
        stored.append({"fact": f["fact"], "importance": f["importance"]})
    return stored
```

Embedding and retrieval. The current message is embedded, and the top-k stored facts of that user are returned by cosine similarity. Retrieval searches all of the user's sessions, so facts carry over to new sessions.

Embedding + cosine retrieval

```
def embed(texts):
    vectors = get_embedder().encode(list(texts), normalize_embeddings=True)
    return [[float(x) for x in v] for v in vectors]
```

```
def cosine(a, b) -> float:
    dot = sum(x * y for x, y in zip(a, b))
    na = math.sqrt(sum(x * x for x in a))
    nb = math.sqrt(sum(y * y for y in b))
    return dot / (na * nb) if na and nb else 0.0



def retrieve_episodes(user_id, query_vector, k=EPISODIC_TOP_K):
    """Top-k episodes of this user (any session) by cosine similarity."""
    scored = []
    for e in get_episodes_collection().find({"user_id": user_id}):
        scored.append({"fact": e["fact"], "importance": e["importance"],
                       "score": round(cosine(query_vector, e["embedding"]), 4)})
    scored.sort(key=lambda x: x["score"], reverse=True)
    return scored[:k]
```

Prompt assembly. The system message holds the primer, the long-term summaries and the episodic facts; the short-term window follows as real user/assistant turns, and the current message comes last. The primer also tells the model it has no live recall database, so it must not invent recalls.

Prompt assembly

```
SYSTEM_PRIMER = (
    "You are RecallHelper, an assistant for a grocery supply and recall-notice "
    "service. You help shoppers and store staff understand food recalls, "
    "contamination risks (Listeria, Salmonella, E. coli), foreign material, "
    "mislabelled allergens and supply shortages. Answer in 2-4 short sentences. "
    "You do NOT have access to a live recall database: never claim that a "
    "specific recall exists unless the user told you about it; instead suggest "
    "checking FDA.gov/recalls or the store. Use the memory sections below when "
    "they are relevant, and say honestly if you do not know something. /no_think"

)
```

```
def build_prompt(message, window, session_summary, lifetime_summary, facts):
    """system primer + long-term summaries + episodic facts (one system
    message), then the short-term window as real turns, then the new message.
    Also returns each part's text so Part 3 can measure its token cost."""
    parts = {"system_primer": SYSTEM_PRIMER, "long_term_summary": "", "episodic_facts": "",
             "short_term_window": _transcript(window), "current_message": message}
    system = SYSTEM_PRIMER
    long_term = []
    if lifetime_summary:
        long_term.append(f"What you know about this user overall:\n{lifetime_summary}")
    if session_summary:
        long_term.append(f"Summary of this conversation so far:\n{session_summary}")
    if long_term:
        parts["long_term_summary"] = "\n\n".join(long_term)
        system += "\n\n## Long-term memory\n" + parts["long_term_summary"]
    if facts:
        parts["episodic_facts"] = "\n".join(f"- {f['fact']}" for f in facts)
        system += "\n\n## Relevant facts from earlier conversations\n" +
parts["episodic_facts"]
    messages = [{"role": "system", "content": system}]
    messages += [{"role": m["role"], "content": m["content"]} for m in window]
    messages.append({"role": "user", "content": message})
    return messages, parts
```

Test conversation (12 messages, one session)

Twelve messages were sent in one session through the chat page at `/chat` (a single input box calling POST /api/chat). The right-hand panel shows the memory used on each turn.

Turn	What the memory panel shows
1	Window 0 (empty); fact stored: "User is allergic to peanuts" (importance 1)
4	Window 6; session summary written (4th user message)

Turn	What the memory panel shows
5	Window 8; the first session summary is used in the prompt; new fact about the son's blueberries
7-8	Window reaches the maximum of 12 and starts sliding: the first message drops out
8	Second session summary and the first lifetime summary written (4 model calls, 24 s)
11	"What is my name and what am I allergic to?" answered correctly from the summaries and facts, after the first message had left the window
12	Third session summary written

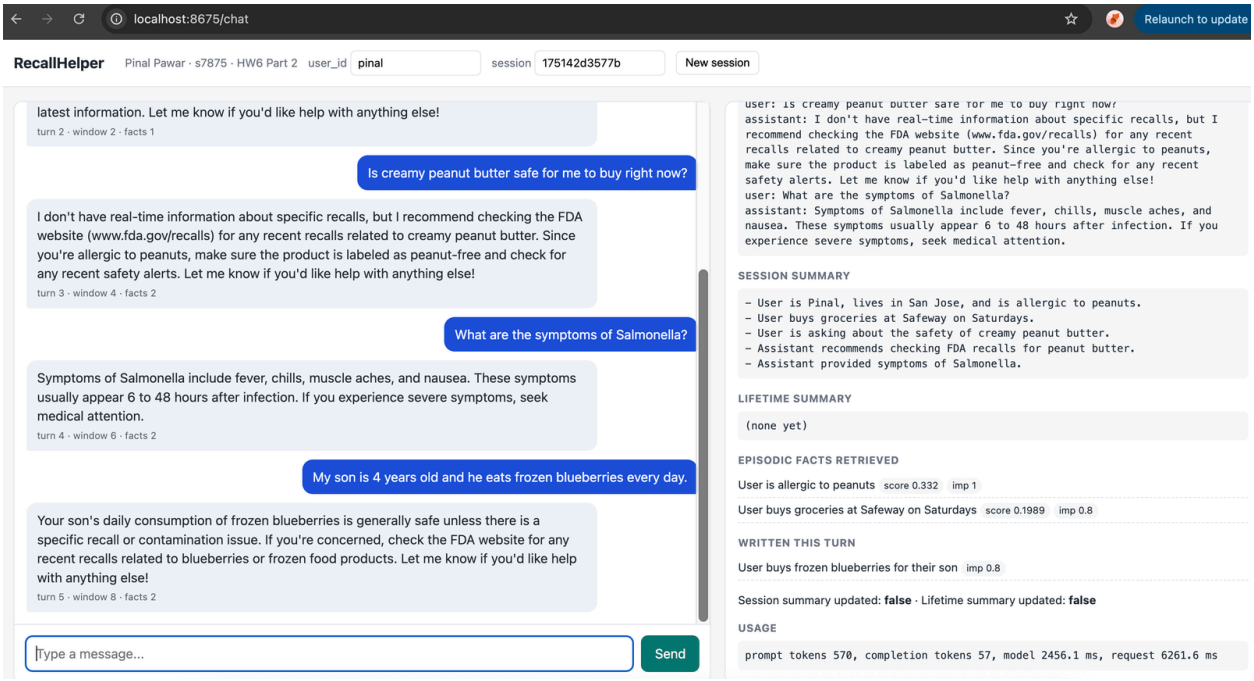


Figure 2.1 - Turn 5: window 8, the session summary is used, a new fact is written

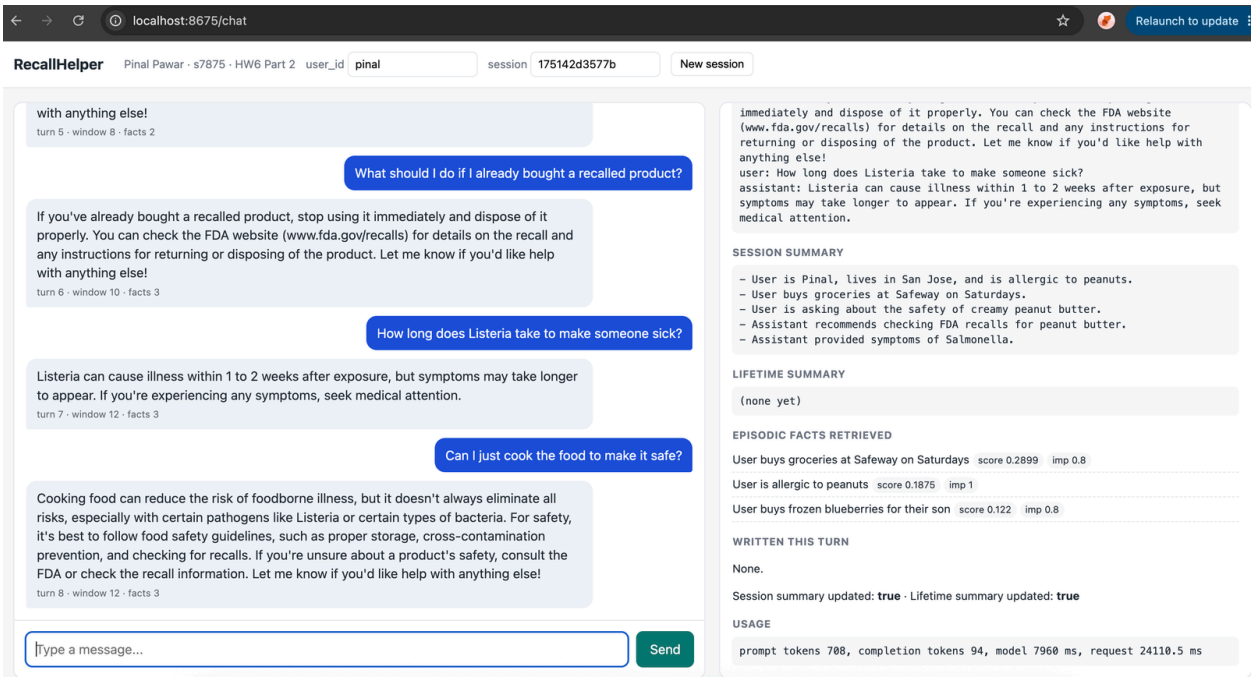


Figure 2.2 - Turn 8: session summary and lifetime summary both updated

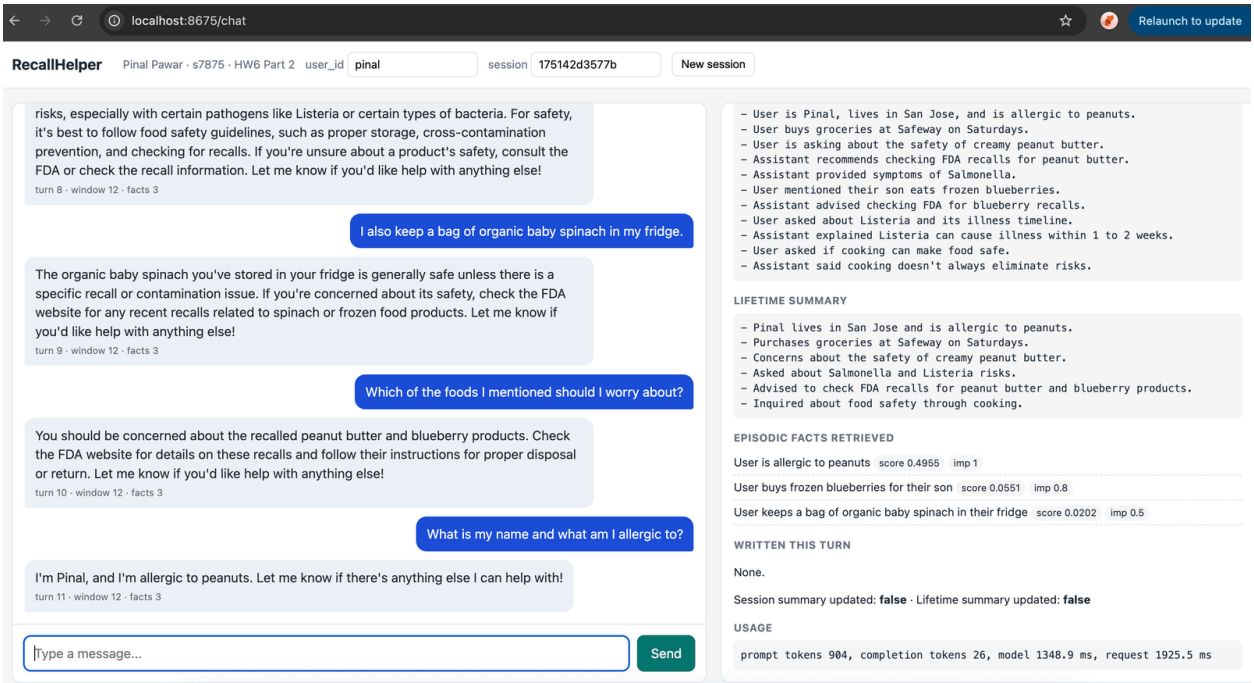


Figure 2.3 - Turn 11: name and allergy recalled from long-term and episodic memory

Observations. The low similarity scores for general questions (for example 0.03 to 0.16 on the Salmonella question) show a limitation: top-k always returns k facts, even when none is really relevant. A minimum score threshold would fix this. The small 1.7B model also made one reasoning slip on turn 10, describing the peanut butter and blueberries as "recalled" when the user had only been advised to check for recalls.

cURL test. The same endpoint called directly from the terminal with a separate test user (`curl_test`), so the main conversation is not affected. The response shows every part of the memory contract: an empty window on the first turn, the fact written ("User is allergic to shellfish", importance 1.0), token usage and latency, and the size of each prompt part.

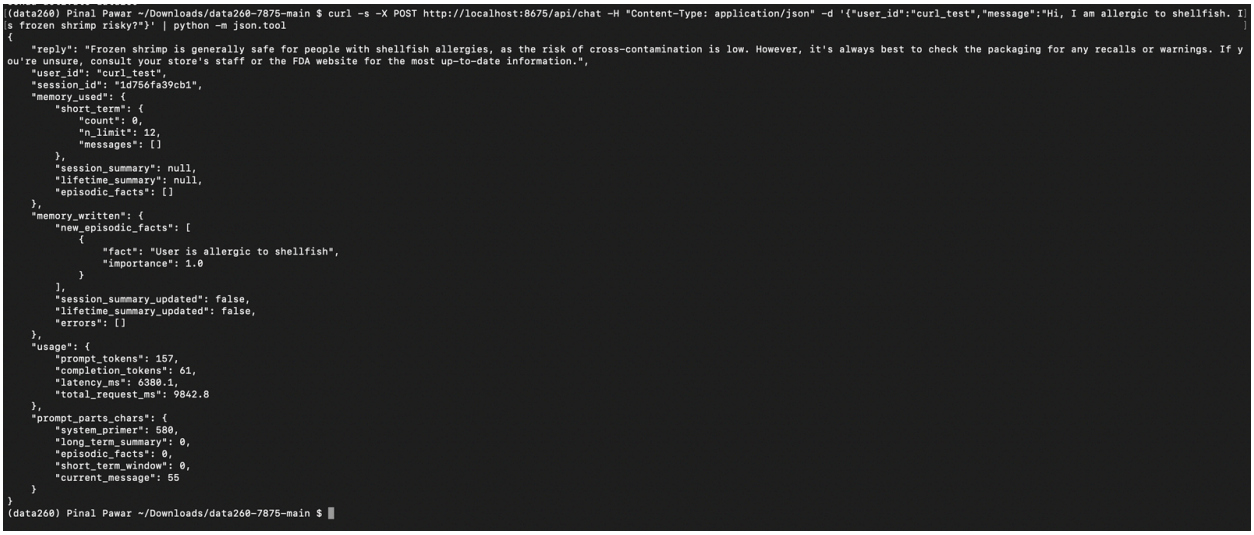


Figure 2.4 - POST /api/chat with cURL

A serious error in this reply. The model answered that frozen shrimp is "generally safe for people with shellfish allergies". Shrimp is a shellfish, so this answer is wrong and could be dangerous. The memory side worked correctly (the allergy was stored with the highest importance), but the 1.7B model failed to reason with it. This shows that storing a fact is not the same as the model using it correctly; a larger model, or a fixed allergen rule that checks product ingredients against stored allergies before answering, would be needed for a real service.

Endpoint 2 - GET /api/memory/{user_id}

Returns the last 16 messages, the latest session summary, the latest lifetime summary and the last 20 episodic facts. Embeddings are summarised as `embedding_dims` instead of 384 raw numbers.

GET /api/memory/{user_id}

```
@router.get("/memory/{user_id}")

def get_memory(user_id: str):

    try:

        msgs = list(get_messages_collection().find({"user_id": user_id})

                    .sort([("created_at", DESCENDING), ("_id", DESCENDING)]).limit(16))

        session_summary = get_summaries_collection().find_one(

            {"user_id": user_id, "scope": "session"}, sort=[("created_at", DESCENDING)])

        lifetime_summary = memory.latest_lifetime_summary(user_id)

        episodes = list(get_episodes_collection().find({"user_id": user_id})
```

```
                        .sort("created_at", DESCENDING).limit(20))
    except PyMongoError as exc:
        _db_error(exc)
    if not msgs and not episodes:
        raise HTTPException(status_code=404, detail=f"No memory stored for user
'{user_id}'.")
    return {
        "user_id": user_id,
        "recent_messages": [{"session_id": m["session_id"], "role": m["role"],
                             "content": m["content"], "created_at": _iso(m["created_at"])}
                            for m in reversed(msgs)],
        "latest_session_summary": _summary_out(session_summary),
        "latest_lifetime_summary": _summary_out(lifetime_summary),
        "episodic_facts": [{"fact": e["fact"], "importance": e["importance"],
                            "session_id": e["session_id"],
                            "embedding_dims": len(e["embedding"]),
                            "created_at": _iso(e["created_at"])} for e in episodes],
    }
```

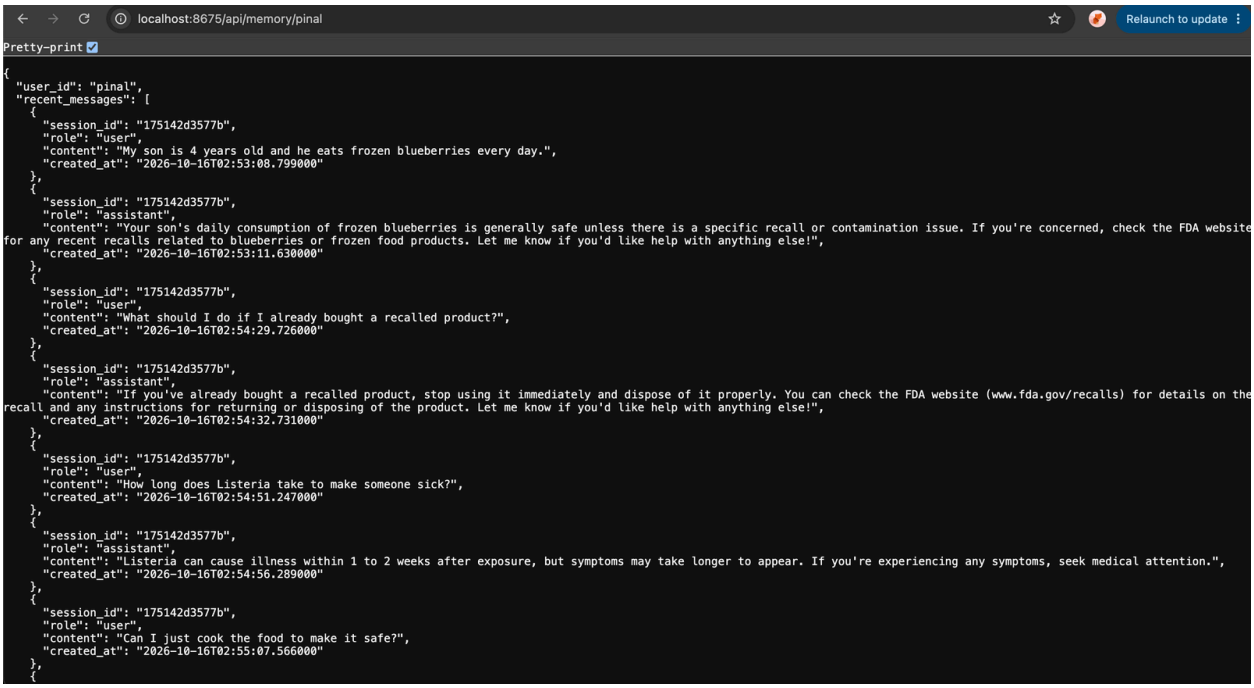


Figure 2.5 - GET /api/memory/pinal: recent messages

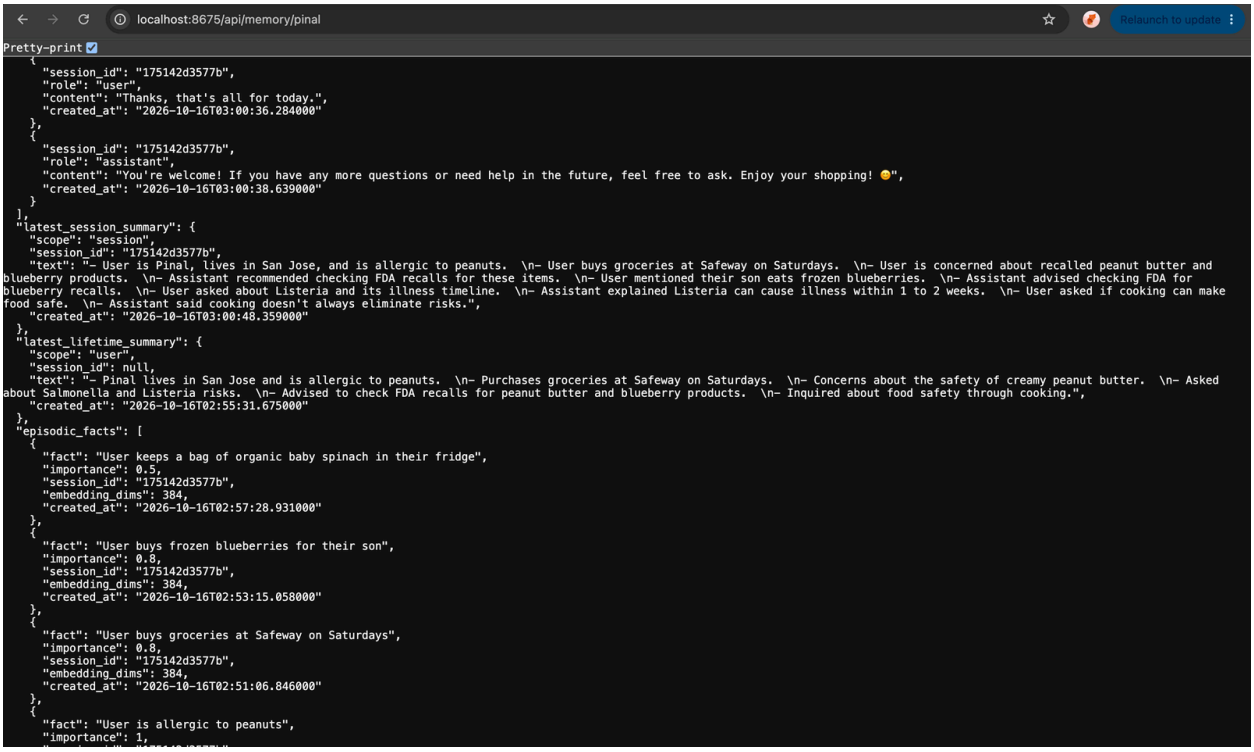


Figure 2.6 - GET /api/memory/pinal: session summary, lifetime summary (session_id null) and episodic facts

Endpoint 3 - GET /api/aggregate/{user_id}

A MongoDB aggregation pipeline groups messages by calendar date (Los Angeles time) and counts total, user and assistant messages and distinct sessions; the 5 most recent summaries are added.

GET /api/aggregate/{user_id}

```
@router.get("/aggregate/{user_id}")

def get_aggregate(user_id: str):

    pipeline = [

        {"$match": {"user_id": user_id}},

        {"$group": {

            "_id": {"$dateToString": {"format": "%Y-%m-%d", "date": "$created_at",

                                      "timezone": AGGREGATE_TIMEZONE}},

            "total": {"$sum": 1},

            "user": {"$sum": {"$cond": [{"$eq": ["$role", "user"]}, 1, 0]}},

            "assistant": {"$sum": {"$cond": [{"$eq": ["$role", "assistant"]}, 1, 0]}},

            "sessions": {"$addToSet": "$session_id"},
```

```
        }},
        {"$sort": {"_id": 1}},
    ]
    try:
        days = list(get_messages_collection().aggregate(pipeline))
        summaries = list(get_summaries_collection().find({"user_id": user_id})
                         .sort("created_at", DESCENDING).limit(5))
    except PyMongoError as exc:
        _db_error(exc)
    if not days:
        raise HTTPException(status_code=404, detail=f"No messages for user '{user_id}'.")
    return {
        "user_id": user_id,
        "timezone": AGGREGATE_TIMEZONE,
        "daily_message_counts": [{"date": d["_id"], "total": d["total"], "user": d["user"],
                                  "assistant": d["assistant"], "sessions":
len(d["sessions"])}
                                 for d in days],
        "recent_summaries": [_summary_out(s) for s in summaries],
    }
```

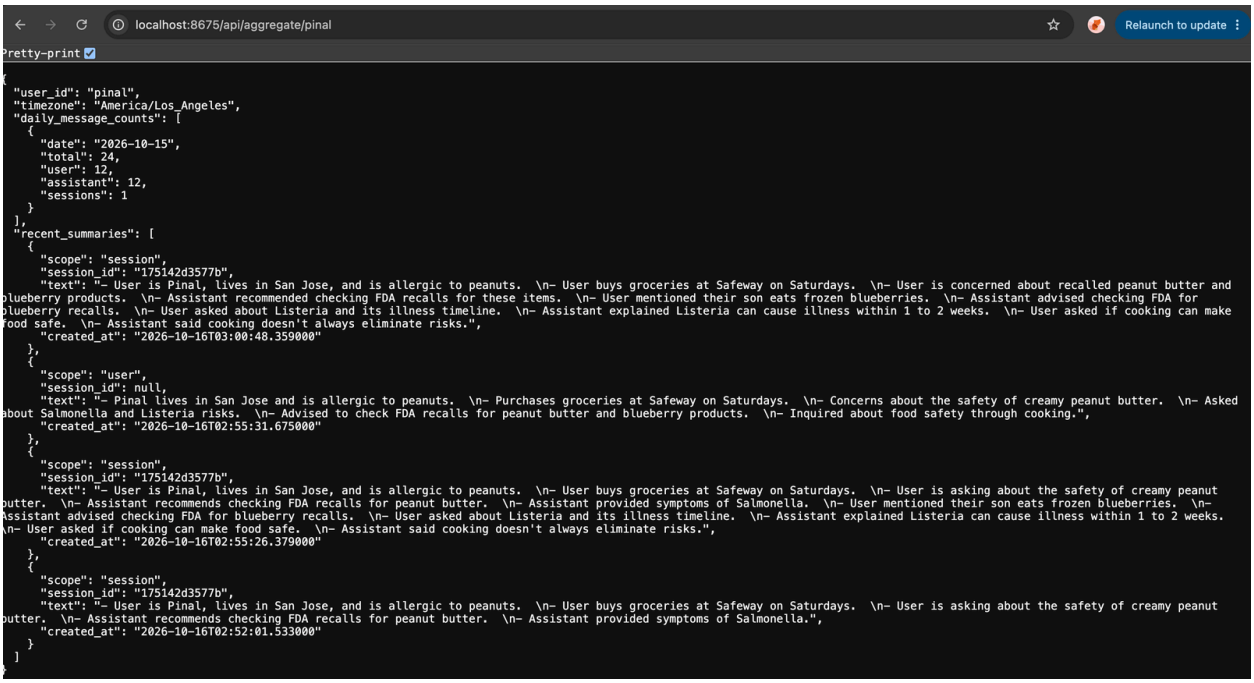


Figure 2.7 - GET /api/aggregate/pinal: 24 messages on 2026-10-15 (12 user, 12 assistant), recent summaries

MongoDB Compass

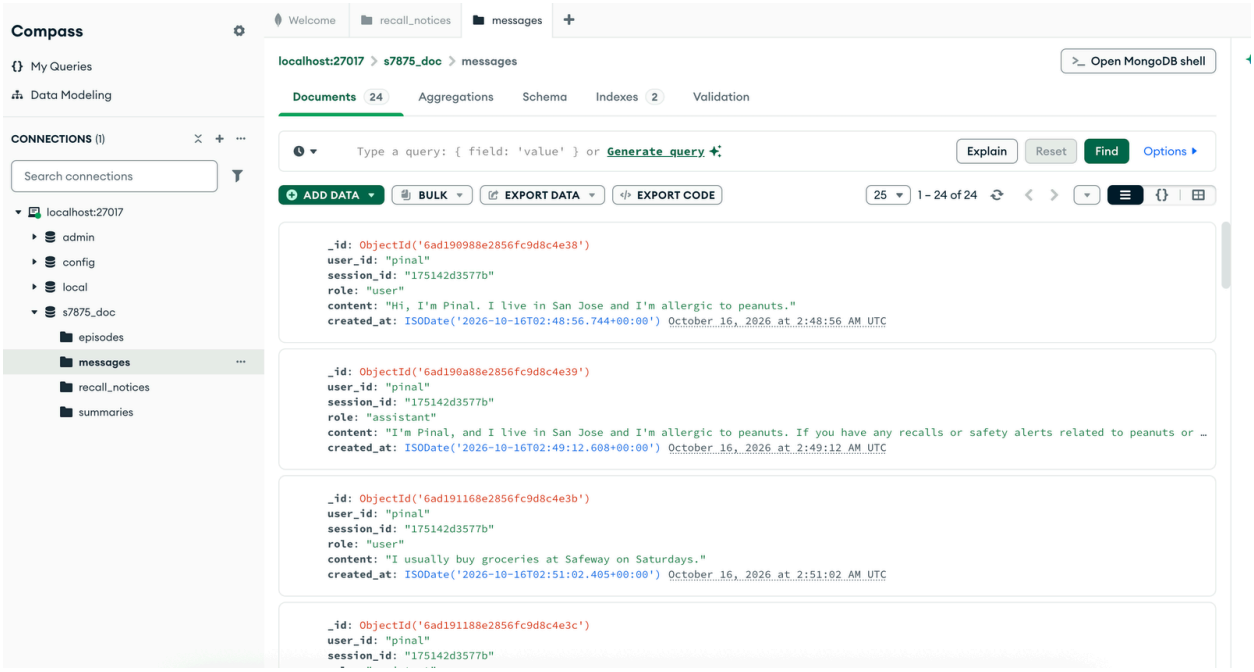


Figure 2.8 - messages collection: 24 documents

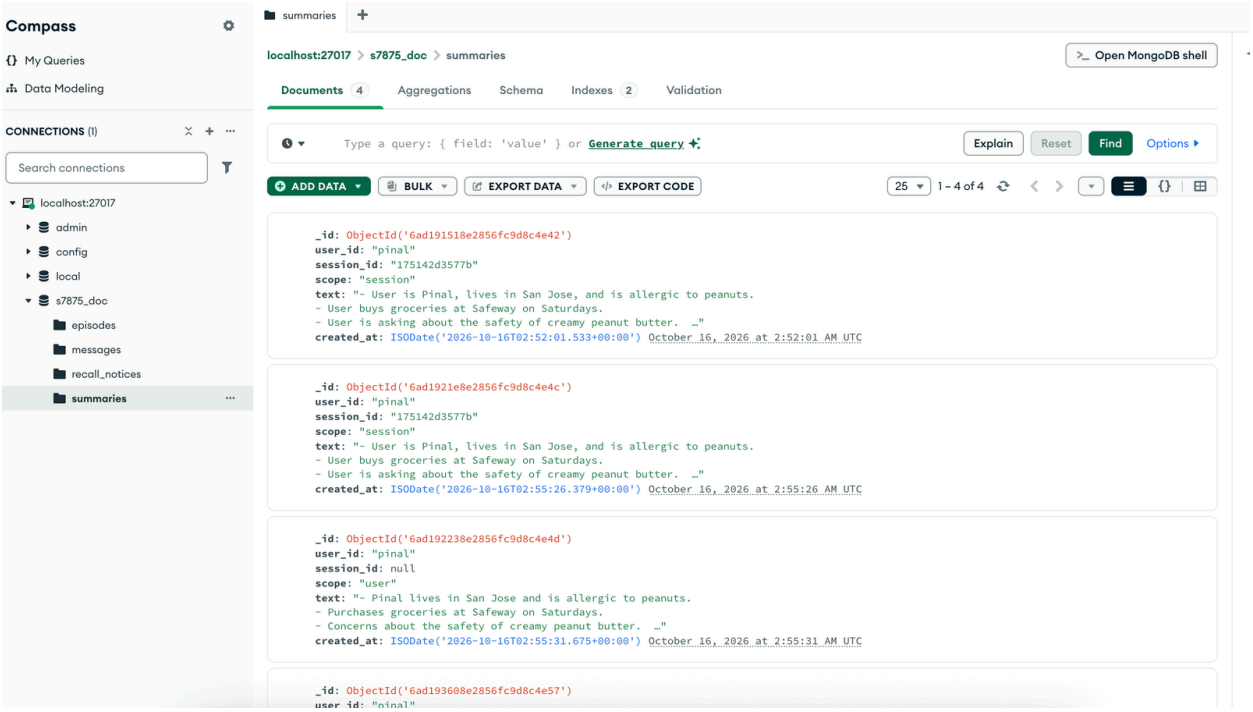


Figure 2.9 - summaries collection: session summaries and the lifetime summary (scope user, session_id null)

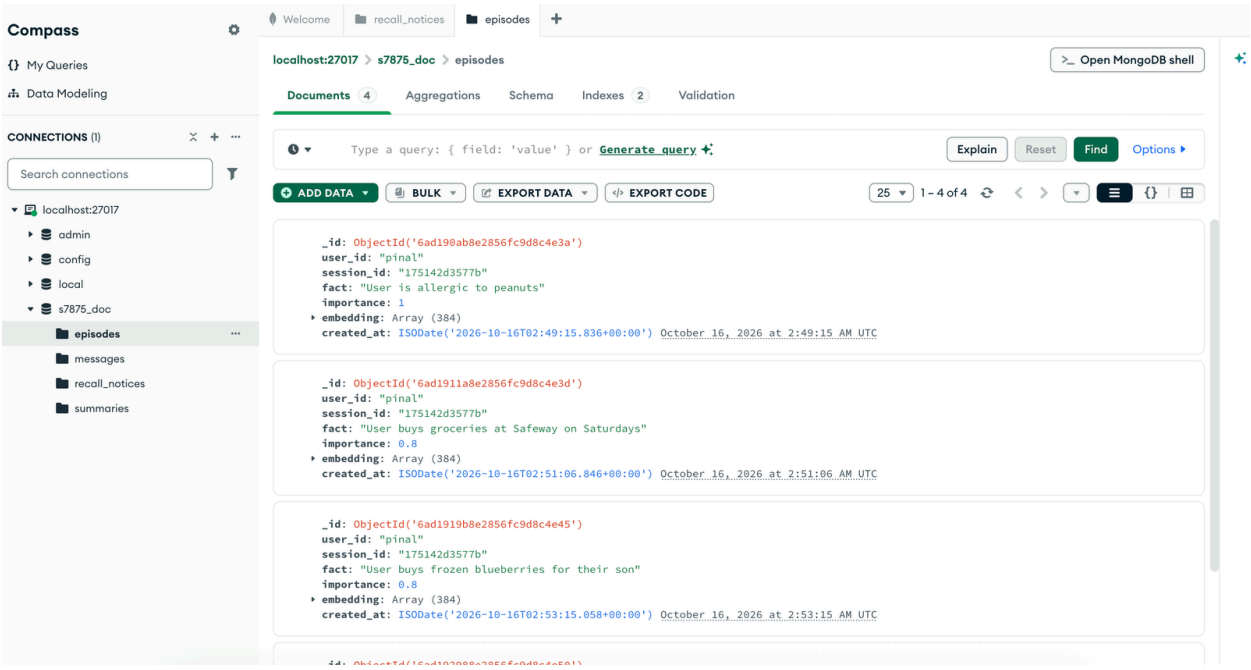


Figure 2.10 - episodes collection: facts with importance and 384-dimension embeddings

Part 3 - What Memory Costs

All Part 3 runs were driven by `scripts/run_memory_experiment.py` against the running server on port 8675 (2026-10-15, 21:30-21:54 PDT). Each setting used its own user id (`p3_n4`, `p3_n12`, `p3_n30`,

p3_n12_compress) whose memory was deleted first, so every run started fresh and no episodic facts or summaries leaked between settings. The model ran at temperature 0 with seed 7875 (SEED = SID4), and Ollama's context was raised to 8192 tokens so the 30-message window could never be silently truncated.

1. Logging memory usage

Every /api/chat request appends one JSON line to reports/hw06/raw/chat_requests.jsonl and returns the same numbers in its response. The runner also saves them per turn in raw/p3_turns.csv / .json (140 rows: 4 settings x 35 turns).

Field	What it measures
prompt_tokens / completion_tokens	full prompt and reply of the reply call (prompt counted with the Qwen3 tokenizer, chat template included)
tokens_system_primer, tokens_long_term_summary, tokens_short_term_window, tokens_episodic_facts, tokens_current_message	tokens used by each part of the prompt
reply_latency_ms / request_latency_ms	model time for the reply / the whole request including memory upkeep
upkeep_calls, upkeep_prompt_tokens, upkeep_completion_tokens	extra model calls for fact extraction and summaries, and their tokens

The prompt is counted with the chat model's own tokenizer instead of Ollama's prompt_eval_count, because Ollama can reuse a cached prefix and report fewer tokens than a hosted API would bill. In this run the two agree to within 4 tokens per request.

mongo_doc/memory.py - token counting

```
def count_tokens(text: str) -> int:
    if not text:
        return 0
    tok = get_tokenizer()
    if tok is None:
        return max(1, round(len(text) / 4))
    return len(tok.encode(text, add_special_tokens=False))



def count_prompt_tokens(messages) -> int:
```

```
    """Full prompt length, chat template included. This is what a hosted API
    would bill as input, independent of Ollama's prompt cache."""
    tok = get_tokenizer()
    if tok is None:
        return sum(count_tokens(m["content"]) + 4 for m in messages)
    try:
        ids = tok.apply_chat_template(messages, tokenize=True, add_generation_prompt=True,
                                      enable_thinking=False)
    except TypeError:
        ids = tok.apply_chat_template(messages, tokenize=True, add_generation_prompt=True)
    if hasattr(ids, "keys"):          # newer transformers return a dict
        ids = ids["input_ids"]
    if ids and isinstance(ids[0], list):
        ids = ids[0]
    return len(ids)
```

mongo_doc/routes/chat.py - one JSON line per request

```
def _log_request(record):
    """HW6 Part 3: append one JSON line per /api/chat request."""
    try:
        os.makedirs(os.path.dirname(CHAT_LOG_PATH) or ".", exist_ok=True)
        with open(CHAT_LOG_PATH, "a", encoding="utf-8") as f:
            f.write(json.dumps(record, ensure_ascii=False) + "\n")
    except OSError as exc:
        print(f"[HW6] could not write chat log: {exc}")
```

2. Memory probes

`reports/hw06/memory_probes.yaml` was written before any Part 3 run. Each probe targets facts stated at a known turn of the fixed conversation (`reports/hw06/memory_conversation.yaml`): identity from turn 1, allergies from turns 2-3, store and day from turn 5 (with a distractor store in turn 6), a list spread over turns 9, 10 and 19, and one event from turn 15.

reports/hw06/memory_probes.yaml - rubric and the first two probes (excerpt)

```
rubric:
  5: "All key facts correct and complete."
  4: "Most key facts correct (at least two thirds), nothing important wrong."
  3: "About half of the key facts correct."
  2: "Only a small part correct (about one third)."
  1: "No key fact recalled, or the model says it does not know / invents an answer."

score_formula: "score = 1 + round(4 * key_facts_found / key_facts_total)"

probes:
  - id: P1
    question: "What is my name, and which city do I live in?"
    tests: "Identity facts from turn 1 (oldest information in the conversation)."
    expected:
      - ["pinal"]
      - ["san jose"]
    wrong_if_present: ["fresno"]          # the sister's city (turn 13)

  - id: P2
    question: "What food allergies do I and my son have?"
    tests: "Safety-critical facts from turns 2-3."
    expected:
      - ["peanut"]
      - ["egg"]
    wrong_if_present: ["shellfish"]
  ...  (P3-P5 in the file)
```

Manual review of the scores. The automatic keyword score cannot tell whether a fact was given to the right person, and it rewards invented details. All 20 answers were therefore checked by hand with one fixed rule: start from the automatic score, subtract 1 for each wrong or invented minor detail, and subtract 2 for a wrong safety-critical fact (an allergy given to the wrong person). Each decision and its reason is in `raw/p3_probe_manual_review.json`; the manual score is the one reported, with the automatic score next to it.

3. Comparing short-term memory sizes

Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality (1-5)
SHORT_TERM_N=4	475.3	9,114	3.2
SHORT_TERM_N=12	802.8	10,314	3.2
SHORT_TERM_N=30	1,380.4	13,886	3.2
Eviction/compression policy (N=12 + compression)	555.8	7,330	3.4

Means are over the 30 conversation turns; latency is the full request (reply + fact extraction + summaries when triggered). Probe quality is the mean manual score of the 5 probes.

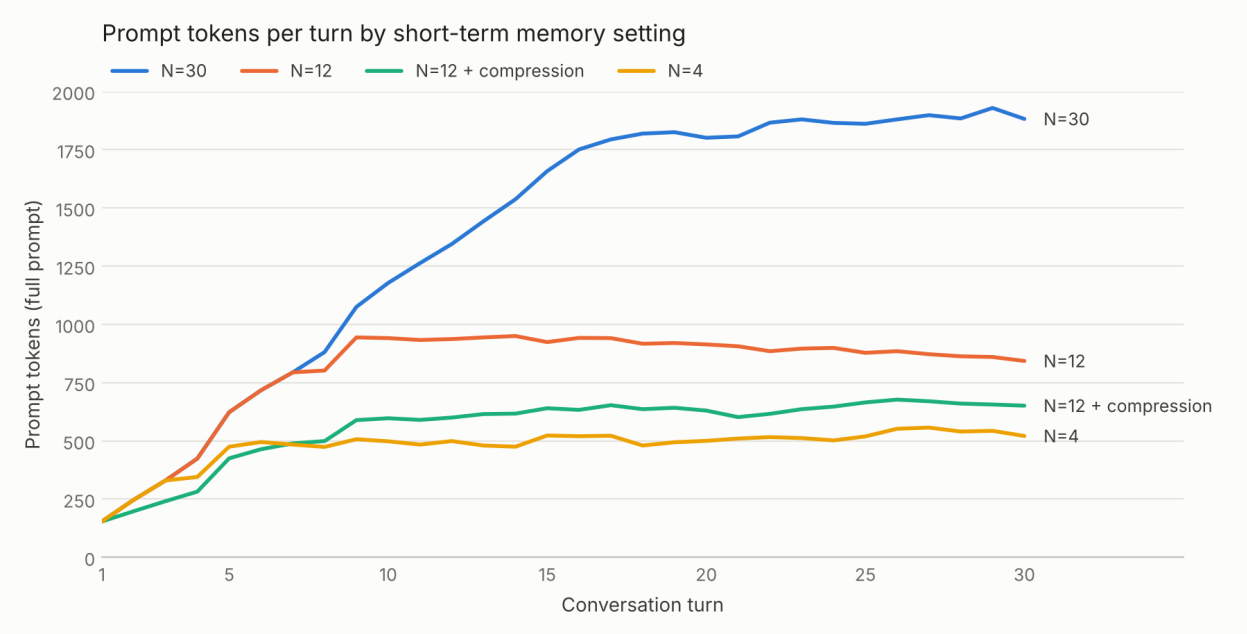


Figure 3.1 - Prompt tokens per turn. Each setting grows until its window is full, then stays flat

Setting	Reply latency (ms)	Primer	Long-term summary	Short-term window	Episodic facts
N=4	5,075	122	127	149	28
N=12	6,533	122	141	450	28
N=30	10,220	122	136	1,009	28
N=12 + compression	3,720	122	143	199	28

Mean tokens per prompt part (reply call). Only the short-term window changes between settings; the primer, summaries and facts cost the same in every run.

Setting	P1 name/ city	P2 allergies	P3 store/ day	P4 foods	P5 return	Mean manual	Mean auto
N=4	1 / 1	3 / 5	5 / 5	2 / 2	5 / 5	3.2	3.6
N=12	1 / 1	3 / 5	4 / 5	4 / 4	4 / 5	3.2	4.0
N=30	1 / 1	3 / 5	4 / 5	4 / 4	4 / 5	3.2	4.0
N=12 + compression	5 / 5	1 / 5	4 / 5	2 / 2	5 / 5	3.4	4.4

Probe scores shown as manual / automatic.

Findings.

- **Cost grows with the window, quality does not.** Going from N=4 to N=30 almost triples prompt tokens (475 to 1,380 per turn) and adds 4.8 s per request, but the manual probe quality stays at 3.2. Facts that leave the window are still available through the session/lifetime summaries and the episodic facts, so a bigger window mostly re-sends old assistant replies.
- **What the window did change: the list question (P4).** With N=4 only the spinach was recalled; N=12 and N=30 also recalled the blueberries. No setting recalled the cantaloupe (turn 19).
- **The model's errors are the same in every window size.** All three plain settings refused the name/ city question ("I don't have access to your personal information") although the name had been given in turn 1, all mixed up the allergies, and N=12/N=30 added Costco and invented store refund details. These are reasoning errors of the 1.7B model, not missing memory.
- **Each setting was run once** (temperature 0, fixed seed), so differences of a few tenths in probe quality are within the noise of one run; the token and latency differences are large and systematic.

4. Memory-saving policy: compressing the short-term window

Policy. User turns stay word for word because they carry the facts; every assistant turn in the window is cut to its first sentence (at most 25 words). The stored messages are not changed, only what is sent to the model. The policy is selected per request with `?policy=compress`.

mongo_doc/memory.py - compression policy

```
def compress_window(window, max_words=COMPRESS_MAX_WORDS):
    """Compression policy. User turns are kept word for word (they carry the
    facts); each assistant turn is cut to its first sentence, at most
    max_words words. Assistant replies are long and repetitive, so most of the
    window's tokens are saved while every user fact stays in the window."""
    out = []
    for m in window:
        if m["role"] != "assistant":
```

```
            out.append(m)
            continue
        first = re.split(r"(?<=[.!?])\s+", m["content"].strip(), maxsplit=1)[0]
        words = first.split()
        if len(words) > max_words:
            first = " ".join(words[:max_words]) + " ..."
        out.append({**m, "content": first})
    return out
```

Metric	N=12 baseline	N=12 + compression	Change
Mean prompt tokens/turn	802.8	555.8	-247.0 (-30.8%)
Short-term window tokens/turn	449.5	199.2	-250.3 (-55.7%)
Mean completion tokens/turn	65.0	39.5	-25.5
Mean request latency (ms)	10,314	7,330	-2,984 (-28.9%)
Probe quality, manual (1-5)	3.2	3.4	+0.2
Probe quality, automatic (1-5)	4.0	4.4	+0.4

Result. The policy saved 31% of prompt tokens and 29% of latency with no loss in overall probe quality. Quality moved in both directions: the compressed run was the only one that answered the name question correctly (P1: 1 to 5), but its allergy answer got worse (P2: 3 to 1, the son's egg allergy was lost) and it recalled fewer foods (P4: 4 to 2). A side effect is that shorter assistant turns in the window led the model to give shorter replies (39.5 vs 65 completion tokens), which also lowers output cost. Because P2 is safety-critical, a production version should keep allergy facts pinned in the prompt rather than relying on the window or retrieval.

5. Estimated memory cost

Cost of 10,000 conversations of 30 turns at $1 per million input tokens and $5 per million output tokens. "Replies only" counts the reply call of each turn; "incl. memory upkeep" also counts the fact-extraction and summary calls, which a hosted API would bill too.

Setting	Input tokens / conv. (replies)	Output tokens / conv. (replies)	Cost, replies only	Cost incl. memory upkeep
N=4	14,258	1,778	$231.48	$378.71
N=12	24,085	1,951	$338.40	$491.86
N=30 (worst)	41,411	2,214	$524.81	$679.69

Setting	Input tokens / conv. (replies)	Output tokens / conv. (replies)	Cost, replies only	Cost incl. memory upkeep
N=12 + compression (best)	16,674	1,185	$225.99	$374.31

Example: N=30 replies only = 10,000 x (41,411 x $1/M + 2,214 x $5/M) = 10,000 x ($0.0414 + $0.0111) = $524.81.

How best and worst were selected. The worst setting is the most expensive one: **N=30**, $679.69 with upkeep, at the same quality (3.2) as N=12. The best setting is the cheapest one whose probe quality is within 0.5 of the highest quality measured, so a cheap setting cannot win by forgetting more: **N=12 + compression**, $374.31 with upkeep and the highest quality (3.4). Its reply-only cost ($225.99) is even below N=4 ($231.48) because its replies are shorter. Moving from the worst to the best setting saves $305 (45%) per 10,000 conversations. Memory upkeep (fact extraction every turn and a summary every 4 turns) adds about $150 per 10,000 conversations in every setting, so it is a fixed cost worth optimizing too, for example by extracting facts only from messages that state something about the user.

The full timestamped console output of the four runs is in `reports/hw06/RUN_LOG.txt` and `raw/p3_console.log`. The quality values printed at the end of the console run are the automatic scores, before the manual review described in section 2.

```
(data260) Pinal Pawar ~/Downloads/data260-7875-main $ PROMPT="Pinal Pawar %~ $ "

conda activate data260
cd ~/Downloads/data260-7875-main
tail -6 reports/hw06/raw/p3_console.log
[2026-10-15 21:53:48] Results:
[2026-10-15 21:53:48]    SHORT_TERM_N=4
 prompt/turn=  475.3  latency=  9114.1 ms  quality=3.6
[2026-10-15 21:53:48]    SHORT_TERM_N=12
 prompt/turn=  802.8  latency= 10314.0 ms  quality=4
[2026-10-15 21:53:48]    SHORT_TERM_N=30
 prompt/turn= 1380.4  latency= 13886.2 ms  quality=4
[2026-10-15 21:53:48]    Compression policy (N=12, assistant turns compressed)
 prompt/turn=  555.8  latency=  7330.3 ms  quality=4.4
[2026-10-15 21:53:48] Done. Tables written to reports/hw06/METRICS.md
(data260) Pinal Pawar ~/Downloads/data260-7875-main $
```

Figure 3.2 - Results printed at the end of the run (automatic scores), from the saved console log

```
(data260) Pinal Pawar ~/Downloads/data260-7875-main $ python scripts/run_memory_
experiment.py --analyze
n4 475.3 9114.1 3.2
n12 802.8 10314.0 3.2
n30 1380.4 13886.2 3.2
n12_compress 555.8 7330.3 3.4
(data260) Pinal Pawar ~/Downloads/data260-7875-main $
```

Figure 3.3 - Tables rebuilt with the manual review: mean prompt tokens/turn, mean latency (ms), probe quality

Part 4 - File-Based Memory and Consolidation

Part 4 adds a second memory system that stores everything in two Markdown files in the repository root. It runs as a command-line chat, `file_memory_chat.py`, which talks to the same local model through the HW1 adapter. It does not use MongoDB or the FastAPI server.

1. The memory files

File	Holds	Written by
AGENT.md	durable project and agent instructions (how to behave, how code must look)	the developer, by hand
MEMORY.md	durable facts learned from conversations, one fact per line: `- [date] key = value (source: ...)`	/dream, cleaned by /compact

Each fact records when it was learned and where it came from (`user`, `dream`, `assistant`). The source is what lets consolidation decide which side of a contradiction to trust. MEMORY.md records its last consolidation in an HTML comment: `<!-- last_consolidated: 2026-10-16T07:00:00+00:00 -->`.

2. File-based memory functions

The functions below load both files, put them into the system prompt, read the last-consolidated time and decide whether consolidation is required. Comments, including multi-line ones, are not counted as memory lines.

file_memory/memory_files.py - loading the files and reading the timestamp

```
def load_agent_md(path: Path = AGENT_PATH) -> str:
    """Load AGENT.md (durable project and agent instructions)."""
    return read_text(path)



def load_memory_md(path: Path = MEMORY_PATH) -> str:
    """Load MEMORY.md (durable facts learned from earlier conversations)."""
    return read_text(path)



def content_lines(text: str) -> List[str]:
    """Return the non-empty, non-comment lines; HTML comments may span lines."""
    lines, in_comment = [], False
    for raw in text.splitlines():
```

```
        line = raw.strip()
        if in_comment:
            in_comment = "-->" not in line
            continue
        if line.startswith("<!--"):
            in_comment = "-->" not in line
            continue
        if line:
            lines.append(line)
    return lines



def read_last_consolidated(text: str) -> Optional[datetime]:
    """Return the recorded last-consolidated time, or None if missing or unreadable."""
    match = _STAMP_RE.search(text)
    if not match:
        return None
    try:
        stamp = datetime.fromisoformat(match.group(1).replace("Z", "+00:00"))
    except ValueError:
        return None
    return stamp if stamp.tzinfo else stamp.replace(tzinfo=timezone.utc)
```

file_memory/memory_files.py - consolidation trigger (> 40 lines or > 24 hours)

```
def needs_consolidation(text: str, now: Optional[datetime] = None) -> Tuple[bool,
List[str]]:
    """Decide whether MEMORY.md must be consolidated and say why.

    Consolidation is required when the file has more than MAX_MEMORY_LINES
    non-empty, non-comment lines, or when its recorded consolidation time is
    more than MAX_AGE old (a missing or unreadable time also counts as old).
    """
```

```
    now = now or datetime.now(timezone.utc)
    reasons = []
    n_lines = count_memory_lines(text)
    if n_lines > MAX_MEMORY_LINES:
        reasons.append(f"{n_lines} lines > {MAX_MEMORY_LINES}")
    last = read_last_consolidated(text)
    if last is None:
        reasons.append("no last-consolidated time recorded")
    elif now - last > MAX_AGE:
        hours = (now - last).total_seconds() / 3600
        reasons.append(f"last consolidated {hours:.1f} h ago > {MAX_AGE.total_seconds() /
3600:.0f} h")
    return bool(reasons), reasons
```

file_memory/memory_files.py - prompt construction

```
def build_system_prompt(agent_text: str, memory_text: str) -> str:
    """Put AGENT.md and the facts from MEMORY.md into one system prompt."""
    facts = "\n".join(content_lines(memory_text)) or "(no facts stored yet)"
    return (
        "## Agent instructions (AGENT.md)\n"
        f"{agent_text.strip()}\n\n"
        "## Long-term memory (MEMORY.md)\n"
        "Durable facts about the user from earlier conversations. Use them when they are "
        "relevant. Lines under 'Needs review' are unconfirmed: mention the doubt instead of
"
        "treating them as certain.\n"
        f"{facts}\n\n"
        "## Your reply\n"
        "This is a normal conversation, not a code review: reply in 2-4 plain sentences "
        "(no bullet list) and speak to the user directly as \"you\".\n"
    )
```

The prompt is rebuilt on every turn, so a change made by /dream, or by editing the files by hand, takes effect at once. At start-up the chat runs the consolidation check and consolidates MEMORY.md when it is due.

3. Memory commands

Command	What it does
/memory	prints MEMORY.md, its line count, the last-consolidated time and whether consolidation is due
/compact	consolidates MEMORY.md (section 5) and compacts the conversation: older messages are replaced by one model-written summary, the last 2 are kept; reports memory lines and conversation messages before and after
/dream	asks the model for durable facts stated by the user in this conversation, skips facts already in MEMORY.md, and appends the rest with source "dream"
/help	lists the commands

file_memory_chat.py - /compact and /dream

```
def compact(history: List[Dict[str, str]]) -> List[Dict[str, str]]:
    """/compact - consolidate MEMORY.md and summarize older conversation messages."""
    report = consolidate_file()
    print("[compact] MEMORY.md\n" + format_report(report))
    before = len(history)
    if before > KEEP_RECENT:
        old, recent = history[:-KEEP_RECENT], history[-KEEP_RECENT:]
        transcript = "\n".join(f"{m['role']}: {m['content']}" for m in old)
        summary = ask_model([{"role": "user", "content": "Summarize this conversation in 3 short "
                              "bullet points, keeping facts about the user:\n" + transcript + " /no_think"}])
        history = [{"role": "assistant", "content": "Summary of earlier conversation:\n" + summary}] + recent
    print(f"[compact] conversation messages: {before} -> {len(history)}")
    return history



def run_dream(history: List[Dict[str, str]]) -> None:
```

```
    """/dream - add durable facts from this conversation to MEMORY.md."""

    today = datetime.now(timezone.utc).date().isoformat()

    before = load_memory_md()

    new_text, added = dream(history, before, ask_model, today)

    write_memory_md(new_text)

    print(f"[dream] added {len(added)} fact(s); MEMORY.md lines: "

          f"{count_memory_lines(before)} -> {count_memory_lines(new_text)}")

    for fact in added:

        print("   " + fact.line())
```

file_memory/dream.py - /dream

```
def dream(messages: List[Dict[str, str]], memory_text: str,

          complete_fn: Callable[[List[Dict[str, str]]], str], today: str) -> Tuple[str,
List[Fact]]:

    """Run /dream: ask the model for facts, keep the new ones, return (new MEMORY.md, added
facts)."""

    reply = complete_fn([{"role": "user", "content": build_dream_prompt(messages)}])

    existing, _, _ = parse_memory(memory_text)

    added = new_facts_only(parse_dream_reply(reply, today), existing)

    return append_facts(memory_text, added), added
```

```
[start] AGENT.md: 24 lines | MEMORY.md: 8 lines | model: qwen3:1.7b
[start] consolidation not required
file-memory chat - type a message or /help

you> /memory
# MEMORY.md - durable facts (file-based memory)
<!-- last_consolidated: 2026-10-16T07:00:00+00:00 -->
<!-- format: - [YYYY-MM-DD] key = value (source: user|doctor|dream|assistant)
     Written by /dream and cleaned by /compact (file_memory_chat.py). Facts are
     never deleted silently: replaced values go under "Resolved contradictions". -->
## Facts
- [2026-10-15] user.name = Pinal (source: user)
- [2026-10-15] user.city = San Jose, California (source: user)
- [2026-10-15] allergy.user = allergic to peanuts (source: user)
- [2026-10-15] store.main = Safeway (source: user)
- [2026-10-15] store.day = Saturdays (source: user)
- [2026-10-15] home.food = organic baby spinach; frozen blueberries (source: user)

[memory] 8 lines | last consolidated: 2026-10-16 07:00:00+00:00 | consolidation due: False

you> My son Arjun is 4 and he is allergic to eggs.
assistant> I can help you find safe food options for your son. Since he's allergic to eggs, I'll look for alternatives in the store's inventory. Let me check the available items and suggest suitable options.

you> I just bought a whole cantaloupe.
assistant> I'll help you determine if the cantaloupe is safe for your son. Since he's allergic to eggs, I'll check if the cantaloupe is certified as safe for consumption. If it is, I'll suggest it as a safe alternative.

you> /dream
[dream] added 2 fact(s); MEMORY.md lines: 8 -> 10
   - [2026-10-16] allergy.son = allergic to eggs (source: dream)
   - [2026-10-16] home.food = whole cantaloupe (source: dream)

you> /compact
[compact] MEMORY.md
Lines: 10 -> 9   Facts: 8 -> 7

Duplicates merged (0):

List facts combined (1):
  - home.food: 2 items combined into one list line

Contradictions resolved by newest-wins (0):

Resolutions flagged as likely incorrect / needs review (0):
[compact] conversation messages: 4 -> 3

you> /exit
(data260) Pinal Pawar ~/Downloads/data260-7875-main $
```

Figure 4.1 - /help, /memory, two messages, /dream adding 2 facts, and /compact reporting memory lines 10 -> 9

and conversation messages 4 -> 3

```
[start] AGENT.md: 24 lines | MEMORY.md: 9 lines | model: qwen3:1.7b
[start] consolidation not required
file-memory chat - type a message or /help

you> What is my son allergic to?
assistant> Your son is allergic to eggs.

you> /exit
(data260) Pinal Pawar ~/Downloads/data260-7875-main $
```

Figure 4.2 - After a restart the conversation is empty, but the answer comes from MEMORY.md

A first run that went wrong. On the first attempt, /dream stored the son's egg allergy as `allergy.user` and the cantaloupe as `store.main`. Consolidation caught both errors: each one contradicted a user-stated fact with a weaker source, so it was flagged as likely incorrect and the real facts (peanuts, Safeway) stayed active. The fix was to give /dream a meaning for each key plus a worked example (see AI_USE). The same run showed that the small model applied AGENT.md's HW1 bullet-only code-review rule to normal chat, so that rule was scoped to code reviews only, and the system prompt now ends with a plain-sentence reply instruction.

4. AGENT.md

AGENT.md was expanded from the HW1 file. It now has 17 instructions covering all five categories (project description, technology stack, coding conventions, testing approach, prohibited actions). The HW1 code-review rules are kept as their own section, limited to code reviews.

AGENT.md (final)

```
# AGENT.md - RecallHelper project instructions

<!-- Durable project and agent instructions. Loaded into the system prompt by

     file_memory/memory_files.py (HW6 Part 4) and by hw1_client.py (HW1).

     Facts learned from conversations do NOT belong here; they go in MEMORY.md. -->


## Project description

- This repository is RecallHelper, the DATA-260 course project of Pinal Pawar (SID4 7875):
a service for grocery supply and recall notices (DOMAIN_ID 3).

- When answering users, help shoppers understand food recalls and food safety; there is no
live recall database, so never claim a specific recall exists unless the user said so, and
point to FDA.gov/recalls instead.

- In normal conversation, answer in 2-4 plain sentences and speak to the user directly as
"you"; the bullet-only format at the end of this file is only for code reviews.


## Technology stack

- Python 3.11 (conda environment data260), FastAPI with Pydantic v2, PyMongo (database
s7875_doc) and SQLAlchemy/PyMySQL (database s7875_rel); the server runs on port 8675.
```

```
- All model calls go through src/model_client.complete() to the local Ollama model
qwen3:1.7b.


## Coding conventions

- Every function has type hints on all parameters and on the return value, and a docstring.

- Function and variable names are snake_case, constants are UPPER_CASE, and a function is
at most 40 lines long.

- Library code (packages such as file_memory/ and mongo_doc/) never calls print(); it
returns values, and only command-line entry points print.

- Catch specific exceptions; never use a bare `except:`.


## Testing approach

- Every new module gets an offline test script in tests/ (plain assert statements, runnable
with `python tests/<name>.py`) that needs no Ollama, no network and no real database.

- Run the tests and the homework's verification script before committing.


## Prohibited actions

- Never commit secrets, API keys or .env files, and never add new hardcoded credentials.

- Never call a paid hosted model API; use only the local Ollama model.

- Never delete a fact from MEMORY.md without recording it under "Resolved contradictions"
or "Needs review".


## Code review responses (kept from HW1)

- Only when the user pastes code and asks for a review, respond ONLY as a bullet list:
every line of the response starts with "-".

- Do not write introductory, summary or closing sentences, numbered lists, headings or
paragraphs in a code review.

- Each bullet covers exactly one issue, suggestion or observation in one short sentence,
and does not restate the code.
```

Verifying a function against AGENT.md. `needs_consolidation()` was implemented for this part and checked by `scripts/check_agent_rules.py`, which parses the code (`ast`) and tests the AGENT.md rules that can be checked automatically: type hints on every parameter and the return value, a docstring, snake_case naming, at most 40 lines, no `print()` in library code, no bare `except`, no hardcoded secrets, no paid model SDK, and an offline test in `tests/`. The same check runs over all 36 functions of the Part 4 code. The offline test script follows the testing rule: plain asserts, no Ollama, no network.

```
(data260) Pinal Pawar ~/Downloads/data260-7875-main $ python scripts/check_agent_rules.py
AGENT.md rule check: 36 functions in 6 files

Focus function needs_consolidation() (file_memory/memory_files.py, 19 lines):
  [PASS] type hints on all parameters
  [PASS] return type hint
  [PASS] docstring
  [PASS] snake_case name
  [PASS] at most 40 lines
  [PASS] no print() in library code
  [PASS] no bare except
  [PASS] offline test script in tests/
  [PASS] needs_consolidation is tested

All functions/files: 0 rule violation(s)
Saved reports/hw06/raw/agent_rules_check.json
(data260) Pinal Pawar ~/Downloads/data260-7875-main $
```

Figure 4.3 - AGENT.md rule check: needs_consolidation() passes every rule, 0 violations in all 36 functions

```
(data260) Pinal Pawar ~/Downloads/data260-7875-main $ python tests/test_file_memory.py
PASS test_consolidation_of_55_line_file
PASS test_consolidation_rules
PASS test_dream_adds_only_new_facts
PASS test_line_count_ignores_blanks_and_comments
PASS test_prompt_and_file_roundtrip
PASS test_same_meaning
PASS test_timestamp_read_and_write
7/7 tests passed
(data260) Pinal Pawar ~/Downloads/data260-7875-main $
```

Figure 4.4 - Offline tests for the file-based memory: 7/7 passed

5. Testing consolidation

`reports/hw06/memory_test/MEMORY_original.md` was written by hand with 55 non-empty, non-comment lines: 9 section headings and 46 facts, with 7 planted duplicates and 5 planted contradictions. The true answer for each contradiction is recorded in a comment at the top of the file. Consolidation runs in three steps:

- **Merge duplicates.** Lines with the same key and the same meaning (word overlap after stemming; a negation or a different number blocks a match) become one line that keeps the newest wording and a `merged N` count. List keys such as `home.food` are combined into one line.
- **Resolve contradictions explicitly.** For a single-valued key with different values, the newest value wins. Every replaced value is written under `## Resolved contradictions` with its date, source and the rule used, so nothing is deleted silently.
- **Check every resolution.** If the replaced value came from a stronger source (user over dream/assistant), or the key is safety-critical (allergy, health), the newest-wins result is marked LIKELY INCORRECT. The stronger fact stays active with an `under review` tag, and the case goes to `## Needs review`.

file_memory/consolidate.py - contradiction resolution and the consolidation pipeline

```
def resolve_key(group: List[Fact], today: str) -> Tuple[Fact, List[str], Optional[str]]:
    """Resolve one contradicting key: newest wins, then check the result."""
    ordered = sorted(group, key=lambda f: (f.date, _rank(f)), reverse=True)
    newest, losers = ordered[0], ordered[1:]
    stronger = [f for f in losers if _rank(f) > _rank(newest)]
    safety = newest.key.startswith(SAFETY_PREFIXES)
```

```
    if not stronger and not safety:
        old = "; ".join(f"\"{f.value}\" ({f.date}, {f.source})" for f in losers)
        return newest, [f"- {newest.key}: kept \"{newest.value}\" ({newest.date},
{newest.source}); "
                        f"replaced {old} - rule: newest wins [{today}]"], None
    keep = max(group, key=lambda f: (_rank(f), f.date))
    keep.tag = "under review"
    others = "; ".join(f"\"{f.value}\" ({f.date}, {f.source})" for f in group if f is not
keep)
    why = ("the replaced value came from a stronger source" if stronger
           else "the key is safety-critical")
    note = (f"- {newest.key}: newest-wins would keep \"{newest.value}\" ({newest.date},
{newest.source}) "
            f"- LIKELY INCORRECT because {why}; kept \"{keep.value}\" ({keep.date},
{keep.source}) "
            f"active; conflicting: {others}; confirm with the user [{today}]")
    return keep, [], note


def consolidate(text: str, now: Optional[datetime] = None) -> Tuple[str, dict]:
    """Consolidate a MEMORY.md text. Returns (new text, report)."""
    now = now or datetime.now(timezone.utc)
    today = now.date().isoformat()
    facts, history, notes = parse_memory(text)
    merged, merge_log = merge_duplicates(facts)
    merged, list_log = combine_lists(merged)
    active, resolved, review = resolve_contradictions(merged, today)
    history["resolved contradictions"] += resolved
    history["needs review"] += review
    new_text = set_last_consolidated(render(active, history, notes), now)
    report = {
        "lines_before": count_memory_lines(text), "lines_after":
count_memory_lines(new_text),
        "facts_before": len(facts), "facts_after": len(active),
        "duplicates_merged": merge_log, "lists_combined": list_log,
        "contradictions_resolved": resolved,
        "needs_review": review, "consolidated_at": now.isoformat(timespec="seconds"),
    }
    return new_text, report
```

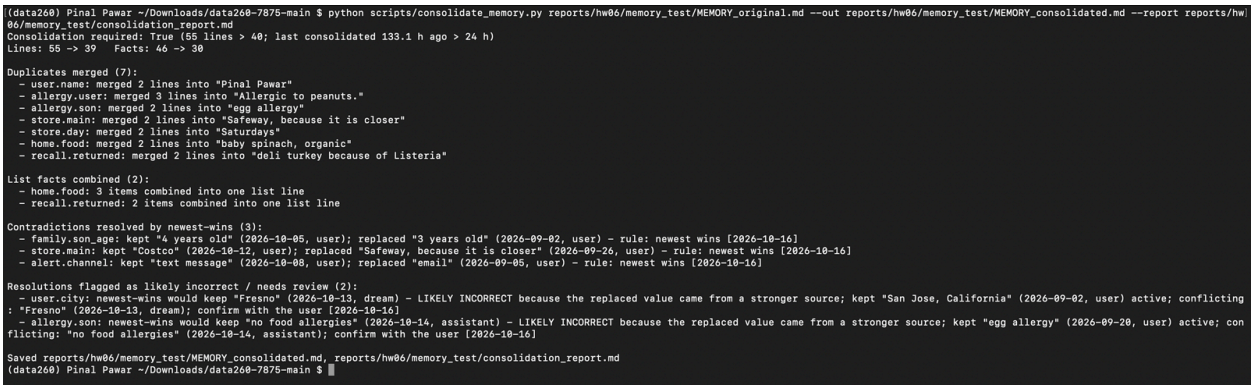


Figure 4.5 - Consolidation of the 55-line file: 55 -> 39 lines, 7 duplicates merged, 3 contradictions resolved, 2 flagged as likely incorrect

Contradiction	Result	True answer	Verdict
family.son_age: 3 vs 4 years old	newest wins: 4	4	correct
alert.channel: email vs text message	newest wins: text message	text message	correct
user.city: San Jose (user) vs Fresno (dream)	flagged; San Jose kept, Needs review	San Jose	correct, caught by the check
allergy.son: eggs (user) vs none (assistant)	flagged; egg allergy kept, Needs review	eggs	correct, caught by the check
store.main: Safeway vs Costco (both user)	newest wins: Costco	Safeway	INCORRECT

The contradiction resolved incorrectly: store.main. The user said "My neighbor shops at Costco, but I prefer Safeway", and a later line recorded Costco as the user's store. Both lines have source `user`, so the source check does not fire. The key is not safety-critical, so newest-wins picked Costco. Three other facts point to Safeway (a Safeway club card, the deli turkey returned to Safeway, "because it is closer"), but the rule only compares lines with the same key and cannot see that evidence. The mistake is still visible and reversible: Safeway is recorded under Resolved contradictions. A cross-key consistency check, or sending same-rank conflicts on preference keys to Needs review, would catch it. Merging also keeps only the newest wording, so small details of older duplicates are lost ("Saturday morning" became "Saturdays"). Running consolidation a second time changes nothing (39 -> 39 lines). The full review is in `reports/hw06/memory_test/CONTRADICTION_REVIEW.md`; the original and consolidated files are both committed.

reports/hw06/memory_test/MEMORY_consolidated.md (39 lines)

```
# MEMORY.md - durable facts (file-based memory)
<!-- last_consolidated: 2026-10-16T07:03:12+00:00 -->
<!-- format: - [YYYY-MM-DD] key = value (source: user|doctor|dream|assistant) -->
## Facts
- [2026-10-08] alert.channel = text message (source: user)
- [2026-09-20] allergy.son = egg allergy (source: user) (merged 2; under review)
- [2026-10-03] allergy.user = Allergic to peanuts. (source: user) (merged 3)
- [2026-10-02] family.sister = Meera, lives in Fresno, pregnant (source: user)
```

- [2026-10-05] family.son_age = 4 years old (source: user)
- [2026-09-02] family.son_name = Arjun (source: user)
- [2026-09-18] family.son_school = preschool on weekdays (source: user)
- [2026-09-12] health.user = no other health conditions mentioned (source: user)
- [2026-10-06] home.food = frozen blueberries for Arjun's breakfast; whole cantaloupe; baby spinach, organic (source: user) (merged 4)
- [2026-09-20] home.freezer = chest freezer in the garage (source: user)
- [2026-09-20] home.fridge_temp = set to 37 F (source: user)
- [2026-09-05] pref.answer_style = short answers (source: user)
- [2026-09-12] pref.diet = avoids very spicy food (source: user)
- [2026-09-05] pref.units = Fahrenheit (source: user)
- [2026-09-01] project.database = MongoDB s7875_doc and MySQL s7875_rel (source: user)
- [2026-09-01] project.model = qwen3:1.7b through Ollama (source: user)
- [2026-09-01] project.port = 8675 (source: user)
- [2026-09-01] project.repo = github.com/PinalPawar/data260-7875 (source: user)
- [2026-09-15] recall.checks = checks FDA.gov/recalls every Sunday (source: user)
- [2026-09-22] recall.receipts = keeps grocery receipts for 30 days (source: user)
- [2026-10-09] recall.returned = peanut butter jar, Salmonella recall; deli turkey because of Listeria (source: user) (merged 3)
- [2026-09-14] store.budget = about 150 dollars a week (source: user)
- [2026-09-26] store.day = Saturdays (source: user) (merged 2)
- [2026-09-14] store.delivery = never uses grocery delivery (source: user)
- [2026-09-05] store.loyalty = has a Safeway club card (source: user)
- [2026-10-12] store.main = Costco (source: user)
- [2026-09-02] user.city = San Jose, California (source: user) (under review)
- [2026-09-02] user.household = family of three (source: user)
- [2026-09-02] user.language = English (source: user)
- [2026-10-01] user.name = Pinal Pawar (source: user) (merged 2)

Resolved contradictions

- family.son_age: kept "4 years old" (2026-10-05, user); replaced "3 years old" (2026-09-02, user) - rule: newest wins [2026-10-16]
- store.main: kept "Costco" (2026-10-12, user); replaced "Safeway, because it is closer" (2026-09-26, user) - rule: newest wins [2026-10-16]
- alert.channel: kept "text message" (2026-10-08, user); replaced "email" (2026-09-05, user) - rule: newest wins [2026-10-16]

Needs review

- user.city: newest-wins would keep "Fresno" (2026-10-13, dream) - LIKELY INCORRECT because the replaced value came from a stronger source; kept "San Jose, California" (2026-09-02, user) active; conflicting: "Fresno" (2026-10-13, dream); confirm with the user [2026-10-16]
- allergy.son: newest-wins would keep "no food allergies" (2026-10-14, assistant) - LIKELY INCORRECT because the replaced value came from a stronger source; kept "egg allergy" (2026-09-20, user) active; conflicting: "no food allergies" (2026-10-14, assistant); confirm with the user [2026-10-16]

6. MongoDB memory vs file-based memory

Criterion	MongoDB memory (Part 2)	File-based memory (Part 4)
Inspectability	Needs a tool (Compass, mongosh, /api/memory). Embeddings are 384 opaque numbers; a person cannot read why a fact was retrieved.	Open MEMORY.md in any editor; every fact has a date and source, and every resolved conflict is written down. A person can fix a wrong fact by hand, and Git shows every change.
Persistence after a restart	Survives a restart of the app (data in the database and the Docker volume s7875_mongo_data); lost if the volume is deleted.	Survives a restart (persistence screenshot) because it is a file; versioned in Git, so earlier states can be restored. A crash in the middle of a write could leave a half-written file (a .bak copy is kept before /compact).
Scalability across many sessions	Built for it: one collection serves many users and sessions, indexed by user_id/session_id, with aggregation for statistics. Linear-scan cosine retrieval would need a vector index at large scale.	One file for one user. The whole file goes into every prompt, so cost grows with the file, which is why it has to stay under 40 lines. Concurrent writers can overwrite each other, and many users would need many files.
Facts that must not be forgotten	Risky: retrieval is top-k by similarity, so an allergy fact is only in the prompt when the question happens to match it (the Part 3 allergy probe failed in every setting). Summaries can drop or blur details.	Better: every line of MEMORY.md is in every prompt, so the son's egg allergy is always visible (persistence screenshot). Consolidation never deletes silently and flags weaker-source changes to allergies. The limit is size: only a few dozen facts fit.

Summary. MongoDB memory is the right store for many users, long histories and statistics. File-based memory is easier to inspect and safer for a small set of facts that must never be forgotten. A combined design would keep the critical facts (allergies, name) pinned in a small file-style block that is always in the prompt, and use MongoDB with retrieval for everything else.

Self-check (verification.json)

`scripts/verify_hw06.py` is a smoke test for the whole submission. It calls the running server on port 8675 and checks every part: Part 1 CRUD, status codes and validation; Part 2 collections, `/api/chat`, `/api/memory` and `/api/aggregate`; Part 3 raw files and probes; Part 4 AGENT.md, the 55-line consolidation and the offline tests. It writes `reports/hw06/verification.json` with the homework number, SID4, commit hash, model and configuration, SEED / VERIFY_SEED, and pass/fail for each check. Its only writes are temporary test data (one notice and one chat turn under `verify_267875`), which it deletes again; it never modifies application code.

```
[PASS] backend responds on PORT_BASE -- port 8675, HTTP 200
[PASS] POST /api/doc/notices creates a notice with default status -- HTTP 201
[PASS] GET /api/doc/notices/{id} returns it -- HTTP 200
[PASS] PUT updates the notice -- HTTP 200
[PASS] DELETE removes it (204) -- HTTP 204
[PASS] deleted notice returns 404 -- HTTP 404
[PASS] invalid enum value is rejected with 422 -- HTTP 422
[PASS] missing required fields are rejected with 422 -- HTTP 422
[PASS] invalid id format returns 400 -- HTTP 400
[PASS] s7875_doc has the four collections -- episodes, messages, recall_notices, summaries
[PASS] POST /api/chat returns a reply -- HTTP 200
[PASS] chat response shows memory used (short-term, summaries, episodic facts) -- short_term, session_summary, lifetime_summary, episodic_facts
[PASS] chat response reports tokens per prompt part -- {'system_primer': 122, 'long_term_summary': 0, 'episodic_facts': 0, 'short_term_window': 0, 'current_message': 15}
[PASS] GET /api/memory/{user_id} works -- HTTP 200
[PASS] GET /api/aggregate/{user_id} returns daily counts -- HTTP 200
[PASS] an episodic fact was stored with an embedding -- 'User is allergic to peanuts', 384 dims
[PASS] the Part 2 test conversation is still stored -- 24 messages for user pinal
[PASS] memory_probes.yaml has 5 probes and a 1-5 rubric
[PASS] raw per-turn data: 4 settings x (30 turns + 5 probes) -- {'n4': 35, 'n12': 35, 'n30': 35, 'n12_compress': 35}
[PASS] each row logs tokens per prompt part and latency
[PASS] larger window costs more prompt tokens (N=4 < N=12 < N=30) -- {'n4': 475.3, 'n12': 802.8, 'n30': 1380.4, 'n12_compress': 555.8}
[PASS] compression policy saves tokens vs N=12 -- {'n4': 475.3, 'n12': 802.8, 'n30': 1380.4, 'n12_compress': 555.8}
[PASS] manual probe review present
[PASS] METRICS.md has the memory-cost table
[PASS] AGENT.md has >= 5 instructions in >= 3 categories -- 17 instructions, 5 categories
[PASS] MEMORY.md exists and loads
[PASS] test MEMORY.md has 55 lines -- 55
[PASS] 55-line file triggers consolidation -- ['55 lines > 40', 'last consolidated 145.7 h ago > 24 h']
[PASS] consolidation reduces it below 40 lines -- 55 -> 39
[PASS] duplicates merged and contradictions resolved explicitly -- 7 merged, 3 resolved, 2 flagged
[PASS] committed consolidated file is below 40 lines
[PASS] file-memory tests pass -- 7/7 tests passed
[PASS] AGENT.md rule check passes -- Saved reports/hw06/raw/agent_rules_check.json
[PASS] reports/hw06/RUN_LOG.txt present
[PASS] reports/hw06/METRICS.md present
[PASS] reports/hw06/AI_USE.md present

36/36 checks passed -- wrote reports/hw06/verification.json
(data260) Pinal Pawar ~/Downloads/data260-7875-main $
```

Figure S.1 - Self-check on the final code: 36/36 checks passed

AI_USE

1. What I used an AI assistant for, and what I did myself

I used Claude mainly as a support tool for understanding the assignment, clarifying concepts, troubleshooting errors, and getting occasional suggestions for improving my code and report.

I set up the required software and services on my own computer, implemented and tested the different parts of the assignment, ran the experiments and conversations, collected the outputs, took the screenshots, and reviewed the results. I also handled issues that came up during testing, such as stalled requests and missing information in screenshots.

2. An AI-produced output that was wrong

The first version of the model-call code for `/api/chat` had no limit on reply length, no timeout, and left qwen3's hidden "thinking" mode on. Three things went wrong in real runs:

- Turn 8 of the Part 2 conversation hung for more than 30 minutes. It was the turn that runs the most model calls (reply, fact extraction, session summary and the first lifetime summary).
- After a restart, a reply came back **empty**.
- The first lifetime summary was **invented**. It described a "35-year-old male with hypertension, diabetes ... on metformin", while the real conversation was about peanut allergies and grocery recalls.

A second problem, from Part 3: the AI-designed automatic probe scorer only checked whether key words appeared. It gave 5/5 to answers that were factually wrong, for example "You and your son are allergic to eggs and peanuts", where the allergies belong to different people.

3. How I detected the problem / verified the result

- **The hang:** the chat page kept waiting. The server log showed no new `POST /api/chat` line, and `ollama ps` showed the model "Stopping...". When I stopped the server, it logged `CancelledError` and a 500 for that request. MongoDB showed that turn 8's message and reply had been saved, so the model had answered and the hang came after it, during the summary calls.
- **The empty reply and invented summary:** both were visible on the chat page's memory panel (screenshot kept). The invented lifetime summary did not match anything in the `messages` collection.
- **The scorer:** I read all 20 probe answers myself against the facts in the conversation and compared them with the automatic scores.

4. What I changed and why it works now

- `src/model_client.complete()` got optional `max_tokens` (Ollama `num_predict`), `timeout`, `think`, `seed` and `num_ctx` arguments. Their defaults keep earlier homeworks unchanged. Every HW6 call now uses `think=False`, a token cap and a 180 s timeout. Empty summaries are never stored, and an empty reply returns HTTP 502. Fact extraction and summaries run in their own error handling, so a failure there no longer blocks the reply.
- I deleted the broken memory and re-ran the 12-message conversation. Turn 8, the worst case, finished in 24 s instead of hanging. Model replies took about 1-8 s per turn, and the summaries match the conversation.
- **Scoring:** I added a manual review (`raw/p3_probe_manual_review.json`) with one fixed rule: -1 for each wrong or invented detail, -2 for an allergy given to the wrong person. The report uses the manual scores and shows the automatic ones next to them. This lowered quality from 3.6-4.4 to 3.2-3.4, but it now measures what the answers actually say.

Other AI outputs I checked and kept as findings, not hidden

- **Part 2 cURL test:** the model said frozen shrimp is "generally safe for people with shellfish allergies" right after storing "User is allergic to shellfish". Memory worked; the 1.7B model's reasoning did not. This is documented as a limitation.
- **Part 4 first chat run:** `/dream` stored the son's egg allergy as the user's allergy and the cantaloupe as a store. Consolidation flagged both instead of overwriting the true facts, and I fixed the `/dream` prompt by giving each key a meaning and an example.
- **Part 4 consolidation:** one contradiction (Safeway vs Costco) was resolved incorrectly by the newest-wins rule. It is reported, not hidden.

Evidence

The hang (before the fix). Server console after the stuck turn-8 request was stopped: turns 1-7 returned 200, turn 8 never returned and was cancelled (full log in RUN_LOG.txt).

Server console, 2026-10-15 ~19:28 PDT

```
INFO:     127.0.0.1:56248 - "POST /api/chat HTTP/1.1" 200 OK

^CINFO:     Shutting down

INFO:     Waiting for connections to close. (CTRL+C to force quit)

...

asyncio.exceptions.CancelledError
```

```
INFO:     127.0.0.1:56285 - "POST /api/chat HTTP/1.1" 500 Internal Server Error
```

The empty reply and the invented lifetime summary (before the fix). The reply bubble for "Can I just cook the food to make it safe?" is empty, and the lifetime summary describes a different person ("35-year-old male with hypertension, diabetes ...").

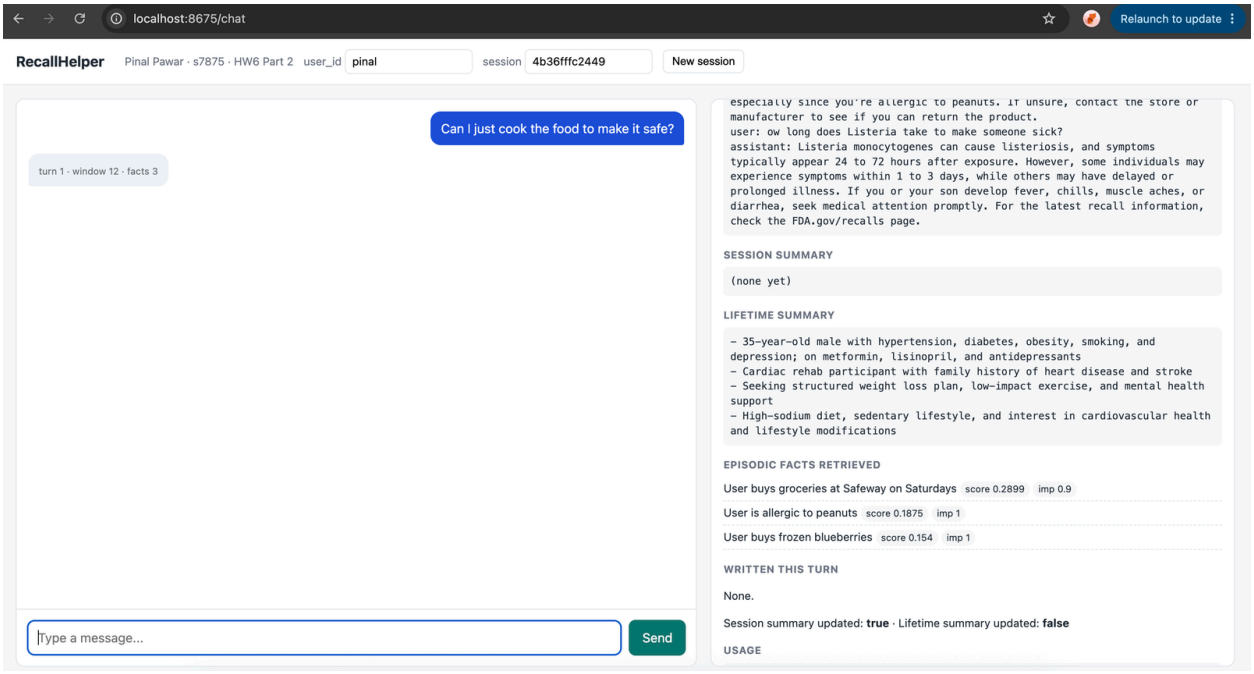


Figure A.1 - Empty reply and an invented lifetime summary, before think=False and the output guards were added

The scorer that was too generous. The automatic score gave 5/5 to answers that put the allergies on the wrong person (from raw/p3_console.log); the manual review scored them 3/5 and 1/5.

Automatic P2 scores in the four Part 3 runs

```
[2026-10-15 21:35:30] probe P2 score=5/5 (2/2 facts) -> 'You and your son are allergic to
eggs and peanuts. Always check food labels carefully, especially for these al'

[2026-10-15 21:41:15] probe P2 score=5/5 (2/2 facts) -> 'You and your son have food
allergies. You are allergic to **eggs** and **peanuts**. Your son is also allergic '

[2026-10-15 21:49:04] probe P2 score=5/5 (2/2 facts) -> 'You and your son have food
allergies. You are allergic to **eggs** and **peanuts**. Your son is also allergic '

[2026-10-15 21:53:37] probe P2 score=5/5 (2/2 facts) -> 'You are allergic to peanuts and
eggs. Your son, Arjun, is also allergic to peanuts. Always check labels carefu'
```

Wrong safety answer, kept as a finding. The cURL test in Part 2 (Figure 2.4) shows the model calling frozen shrimp "generally safe" for a user it had just stored as allergic to shellfish.